

Python para Backend

# Django Rest Framework

---

Javier Villegas

12 de mayo 2025



## Construyendo APIs con Django REST Framework

Este laboratorio te guiará a través de la creación de una API RESTful completa utilizando Django REST Framework (DRF). Trabajaremos con el sistema POS existente para crear endpoints que permitan acceder y manipular datos de artículos, órdenes y usuarios.

Vamos a implementar:

1. Serializers (incluyendo serializers dinámicos y anidados)
2. Vistas basadas en funciones y clases
3. Mixins y vistas genéricas
4. ViewSets y enrutamiento
5. Filtrado y paginación
6. Throttling
7. Autenticación y seguridad
8. Validaciones
9. Versionamiento de APIs
10. Manejo de errores y personalización de respuestas

# Introducción a Django Rest Framework

## ¿Qué es REST?

REST (Representational State Transfer) es un estilo arquitectónico para diseñar aplicaciones en red. Se basa en la comunicación cliente-servidor sin estado, normalmente a través de HTTP.

### Principios clave:

- Sin estado: Cada solicitud contiene toda la información necesaria para procesarla.
- Basado en recursos: Las API se organizan en torno a recursos (entidades como artículos, productos, usuarios)
- Utiliza métodos HTTP estándar: GET, POST, PUT, DELETE, etc.
- Orientado a la representación: Los recursos pueden tener múltiples representaciones (JSON, XML, etc.)

## ¿Qué es Django REST Framework?

Django REST Framework es un potente conjunto de herramientas para construir APIs Web en Django. Proporciona:

- Serialización para convertir modelos Django a JSON/XML
- Autenticación y permisos
- Interfaz API navegable
- Viewsets y routers para un desarrollo rápido
- Amplia documentación y soporte de la comunidad

**Proyecto base:** Continuamos trabajando sobre el mismo proyecto [encomiendas/](#) que construimos en las sesiones anteriores. No se crea un proyecto nuevo. DRF se agrega como una capa encima de los modelos, managers, validators y querysets que ya están implementados.

## PUNTO DE PARTIDA

Para esta sesión necesitas tener listo de sesiones anteriores:

- Proyecto Django corriendo en Docker con PostgreSQL (Sesión 2)
- Modelos: Cliente, Empleado, Ruta, Encomienda, HistorialEstado (Sesión 3)
- Managers personalizados: con\_relaciones(), activas(), pendientes() (Sesión 3)
- Validadores y método cambiar\_estado() en el modelo (Sesión 3)
- Archivos choices.py, validators.py y querysets.py existentes

## Paso 1 — Instalar las dependencias dentro del contenedor Docker

Todos los comandos se ejecutan **dentro del contenedor Docker**, siguiendo el mismo flujo de las sesiones anteriores. Nunca instalamos nada directamente en el sistema operativo local.

None

# 1. Asegúrate de que los contenedores están corriendo

```
docker compose up -d
```

# 2. Verificar que el contenedor web está activo

```
docker compose ps
```

# Deberías ver algo como:

| # NAME              | STATUS  | PORTS                  |
|---------------------|---------|------------------------|
| # encomiendas-web-1 | running | 0.0.0.0:8000->8000/tcp |
| # encomiendas-db-1  | running | 5432/tcp               |

# 3. Instalar las librerías dentro del contenedor

```
docker compose exec web pip install djangoRESTframework
```

```
docker compose exec web pip install djangorestframework-simplejwt
docker compose exec web pip install django-filter
docker compose exec web pip install drf-spectacular
docker compose exec web pip install django-cors-headers
```

## Paso 2 — Actualizar requirements.txt

Después de instalar, actualiza el archivo `requirements.txt` para que cualquier persona que clone el repositorio pueda instalar todas las dependencias con un solo comando:

None

```
# Desde la raíz del proyecto, dentro del contenedor
docker compose exec web pip freeze > requirements.txt

# Verifica que quedaron registradas las nuevas librerías
docker compose exec web pip show djangorestframework

# El requirements.txt ahora incluirá entre otras:
# djangorestframework==3.15.x
# djangorestframework-simplejwt==5.x.x
# django-filter==24.x
# drf-spectacular==0.27.x
# django-cors-headers==4.x.x
```

## Paso 3 — Registrar las apps en settings.py

Abre el archivo `config/settings.py` en tu editor y agrega las nuevas apps a la lista `INSTALLED_APPS`. Este archivo ya existe y tiene las apps de sesiones anteriores (envios, clientes, rutas):

None

```
# config/settings.py

# Archivo existente – solo agregar las líneas marcadas con <-- NUEVO

INSTALLED_APPS = [

    'django.contrib.admin',

    'django.contrib.auth',

    'django.contrib.contenttypes',

    'django.contrib.sessions',

    'django.contrib.messages',

    'django.contrib.staticfiles',

    # Apps del proyecto (ya existían desde sesión 2)

    'envios',

    'clientes',

    'rutas',

    # Nuevas librerías de API (agregar estas 4 líneas) <-- NUEVO

    'rest_framework',

    'rest_framework_simplejwt',

    'django_filters',

    'drf_spectacular',

    'corsheaders',

]
```

## Paso 4 — Agregar la configuración de DRF en settings.py

En el mismo archivo `config/settings.py`, al final del archivo (o junto a las otras configuraciones), agrega el bloque de configuración de DRF:

```
None

# config/settings.py

# Agregar al final del archivo, debajo de las configuraciones existentes


from datetime import timedelta


# — Django REST Framework —————
REST_FRAMEWORK = {
    # Autenticación: JWT por defecto para toda la API
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ],
    # Permisos: requiere autenticación por defecto
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
    # Paginación: 15 registros por página
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 15,
    # Documentación automática con drf-spectacular
    'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',
    # Filtros: django-filter como backend por defecto
```

```

'DEFAULT_FILTER_BACKENDS': [
    'django_filters.rest_framework.DjangoFilterBackend',
    'rest_framework.filters.SearchFilter',
    'rest_framework.filters.OrderingFilter',
],
}

# — JWT: configuración de tokens —————
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60), # token expira en 1 hora
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),    # refresh expira en 7 días
    'ROTATE_REFRESH_TOKENS': True,                  # rotar el refresh en cada
    uso
    'AUTH_HEADER_TYPES': ('Bearer',),               # Authorization: Bearer
    <token>
}

# — CORS: permitir peticiones desde el frontend —————
CORS_ALLOW_ALL_ORIGINS = True # en desarrollo

# En producción reemplazar por CORS_ALLOWED_ORIGINS = ['https://tu-frontend.com']

# — Documentación de la API —————
SPECTACULAR_SETTINGS = {
    'TITLE': 'API Sistema de Encomiendas',

```

## Paso 5 — Agregar CORS al middleware

En el mismo `config/settings.py`, busca la lista `MIDDLEWARE` que ya existe y agrega `'corsheaders.middleware.CorsMiddleware'` como **primera línea** de la lista:

Python

```
# config/settings.py
# MIDDLEWARE ya existe - solo agregar la primera línea

MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware',  # <-- NUEVO: debe ir PRIMERO
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

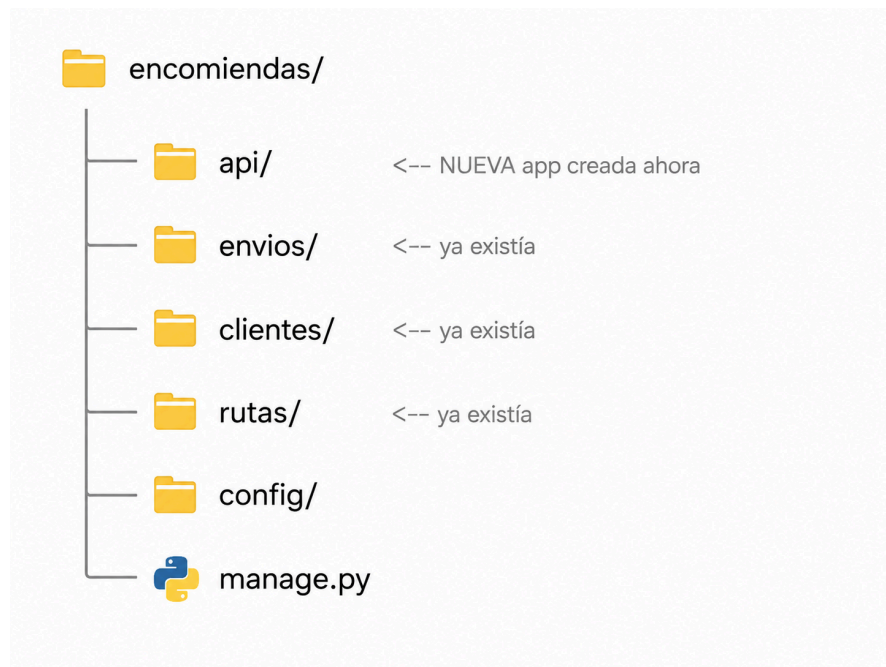
## Paso 6 — Crear la app api y configurar las URLs

Creamos una app dedicada para organizar todos los archivos de la API (serializers, viewsets, filtros, paginación). Esto mantiene el código separado de las apps de dominio (envios, clientes, rutas).

None

```
# Crear la app 'api' dentro del contenedor
docker compose exec web python manage.py startapp api
```

La estructura del proyecto ahora será:



Luego crea el archivo `api/urls.py` (no existe aún, créalo tú):

Python

```
# api/urls.py <-- archivo NUEVO, crearlo desde cero
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from rest_framework_simplejwt.views import (
    TokenObtainPairView,
    TokenRefreshView,
)
from drf_spectacular.views import (
    SpectacularAPIView,
    SpectacularSwaggerView,
)
```

```
router = DefaultRouter()

# Aquí se registrarán los ViewSets en los siguientes pasos
# router.register('encomiendas', EncomiendaViewSet,
# basename='encomienda')

urlpatterns = [

    # Endpoints de autenticación JWT
    path('auth/token/', TokenObtainPairView.as_view(),
name='token_obtain'),
    path('auth/token/refresh/', TokenRefreshView.as_view(),
name='token_refresh'),

    # Documentación interactiva
    path('schema/', SpectacularAPIView.as_view(), name='schema'),
    path('docs/', SpectacularSwaggerView.as_view(),
name='swagger'),

    # URLs del router (ViewSets)
    path('', include(router.urls)),

]
```

## Paso 8 — Verificar que todo funciona

Con los contenedores corriendo, verifica que DRF está funcionando correctamente:

Python

```
# Reiniciar el contenedor para que tome los cambios en settings.py
docker compose restart web

# Verificar que no hay errores de configuración
docker compose exec web python manage.py check
# Salida esperada: System check identified no issues (0 silenced).

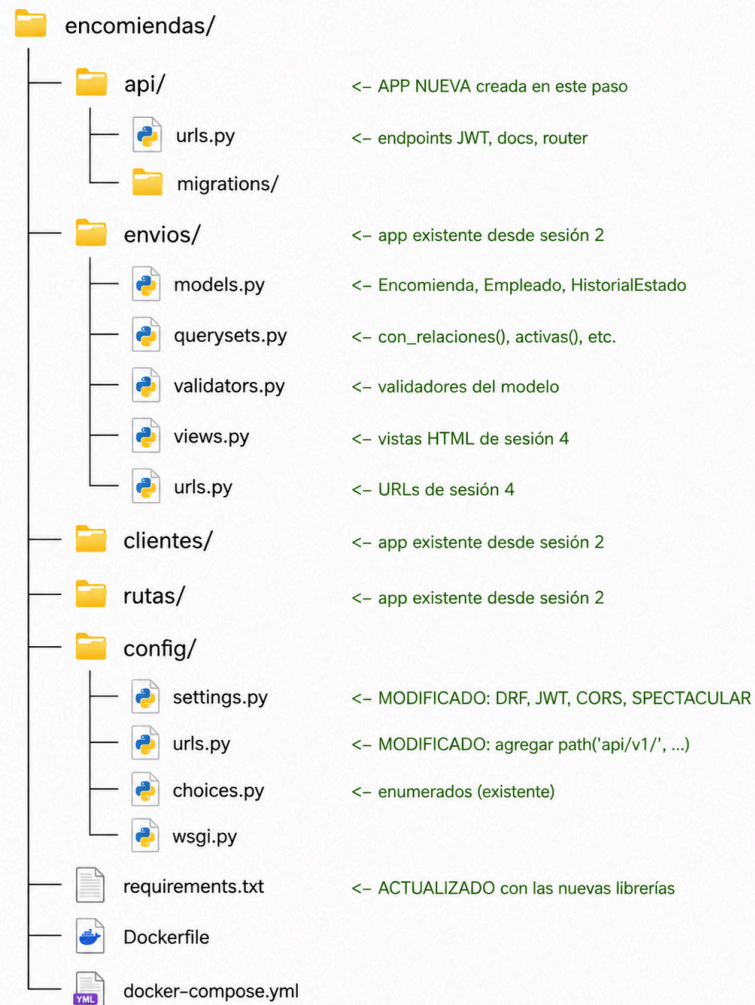
# Verificar que las URLs de la API responden
# Abrir en el navegador:
# http://localhost:8000/api/v1/          <- DRF Browsable API
# (página de bienvenida)
# http://localhost:8000/api/v1/docs/    <- Swagger UI
# (documentación interactiva)
# http://localhost:8000/api/v1/auth/token/ <- Endpoint de login JWT

# Probar el endpoint de login con curl (opcional)
curl -X POST http://localhost:8000/api/v1/auth/token/ \
  -H 'Content-Type: application/json' \
  -d '{"username": "tu_superuser", "password": "tu_password"}'

# Respuesta esperada:
```

```
# {
#   "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
#   "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
# }
```

## ESTRUCTURA DEL PROYECTO AL INICIO DE ESTA SESIÓN



## 5.1. REST in Action

REST (Representational State Transfer) define cómo estructurar la comunicación entre cliente y servidor. Los recursos se identifican con URLs y las operaciones con verbos HTTP.

### Endpoints del sistema de encomiendas

| Método | URL                                      | Acción                  | Status esperado |
|--------|------------------------------------------|-------------------------|-----------------|
| GET    | /api/v1/encomiendas/                     | Listar todas            | 200 OK          |
| POST   | /api/v1/encomiendas/                     | Crear nueva             | 201 Created     |
| GET    | /api/v1/encomiendas/{id}/                | Detalle                 | 200 OK          |
| PUT    | /api/v1/encomiendas/{id}/                | Actualizar completo     | 200 OK          |
| PATCH  | /api/v1/encomiendas/{id}/                | Actualizar parcial      | 200 OK          |
| DELETE | /api/v1/encomiendas/{id}/                | Eliminar                | 204 No Content  |
| POST   | /api/v1/encomiendas/{id}/cambiar_estado/ | Acción personalizada    | 200 OK          |
| GET    | /api/v1/encomiendas/con_retraso/         | Acción de lista         | 200 OK          |
| GET    | /api/v1/clientes/                        | Listar clientes activos | 200 OK          |
| GET    | /api/v1/rutas/                           | Listar rutas activas    | 200 OK          |
| POST   | /api/v1/auth/token/                      | Obtener JWT             | 200 OK          |
| POST   | /api/v1/auth/token/refresh/              | Renovar JWT             | 200 OK          |

## Status codes HTTP más usados

| Código | Nombre                | Cuándo usarlo                            |
|--------|-----------------------|------------------------------------------|
| 200    | OK                    | GET o PUT exitoso                        |
| 201    | Created               | POST exitoso: recurso creado             |
| 204    | No Content            | DELETE exitoso: sin cuerpo de respuesta  |
| 400    | Bad Request           | Datos inválidos: errores de validación   |
| 401    | Unauthorized          | Sin token o token inválido               |
| 403    | Forbidden             | Token válido pero sin permiso            |
| 404    | Not Found             | Recurso no existe                        |
| 405    | Method Not Allowed    | Método HTTP no soportado en ese endpoint |
| 500    | Internal Server Error | Error no controlado en el servidor       |

## 5.2. Serializers y FBV con @api\_view

Los Serializers convierten objetos Python (instancias de modelo) a JSON y viceversa. Son el equivalente de los formularios de Django pero para APIs. El decorador `@api_view` transforma una función en un endpoint REST.

### 5.2.1 ModelSerializer para el sistema de encomiendas

Un `ModelSerializer` en Django REST Framework (DRF) es un atajo para crear clases de serialización que se asignan a los modelos de Django. Reduce considerablemente el código repetitivo al generar automáticamente campos y validadores basados en la definición de tu modelo.

#### Ventajas principales

- Asignación automática de campos: crea automáticamente un conjunto de campos basados en el modelo.
- Validación integrada: genera validadores predeterminados, como restricciones `unique_together`.

- Lógica CRUD predeterminada: incluye implementaciones predeterminadas para los métodos `.create()` y `.update()`.

Python

```
# envios/serializers.py
from rest_framework import serializers
from .models import Encomienda, HistorialEstado, Empleado
from clientes.models import Cliente
from rutas.models import Ruta

class ClienteSerializer(serializers.ModelSerializer):
    # @property del modelo expuesta como campo de solo lectura
    nombre_completo = serializers.ReadOnlyField()
    esta_activo = serializers.ReadOnlyField()

    class Meta:
        model = Cliente
        fields = [
            'id', 'tipo_doc', 'nro_doc',
            'nombres', 'apellidos', 'nombre_completo',
            'telefono', 'email', 'esta_activo',
        ]

class RutaSerializer(serializers.ModelSerializer):
    class Meta:
        model = Ruta
        fields = [
            'id', 'codigo', 'origen', 'destino',
            'precio_base', 'dias_entrega', 'estado',
        ]

class HistorialEstadoSerializer(serializers.ModelSerializer):
    empleado_nombre = serializers.ReadOnlyField(source='empleado.__str__')
    estado_anterior_display = serializers.CharField(
        source='get_estado_anterior_display', read_only=True
    )
```

```

estado_nuevo_display      = serializers.CharField(
source='get_estado_nuevo_display', read_only=True
)

class Meta:
model = HistorialEstado
fields = [
    'id', 'estado_anterior', 'estado_anterior_display',
    'estado_nuevo', 'estado_nuevo_display',
    'empleado_nombre', 'observacion', 'fecha_cambio',
]

```

```

class EncomiendaSerializer(serializers.ModelSerializer):
    # Propiedades del modelo como campos de solo lectura
    esta_entregada      = serializers.ReadOnlyField()
    tiene_retraso      = serializers.ReadOnlyField()
    dias_en_transito    = serializers.ReadOnlyField()
    descripcion_corta   = serializers.ReadOnlyField()

    # Campo calculado con método
    estado_display = serializers.SerializerMethodField()

    class Meta:
model = Encomienda
fields = [
    'id', 'codigo', 'descripcion', 'descripcion_corta',
    'peso_kg', 'volumen_cm3', 'costo_envio',
    'remitante', 'destinatario', 'ruta', 'empleado_registro',
    'estado', 'estado_display',
    'fecha_registro', 'fecha_entrega_est', 'fecha_entrega_real',
    'esta_entregada', 'tiene_retraso', 'dias_en_transito',
    'observaciones',
]
read_only_fields = ['codigo', 'fecha_registro', 'fecha_entrega_real']

def get_estado_display(self, obj):
return obj.get_estado_display()

class EncomiendaDetailSerializer(EncomiendaSerializer):
    """Serializer con objetos anidados completos (para el detalle)"""
    remitente      = ClienteSerializer(read_only=True)
    destinatario   = ClienteSerializer(read_only=True)

```

```

ruta          = RutaSerializer(read_only=True)
historial     = HistorialEstadoSerializer(many=True, read_only=True)

# Para escritura: aceptar solo el ID
remitente_id = serializers.PrimaryKeyRelatedField(
    queryset=Cliente.objects.ativos(),
    write_only=True, source='remitente'
)
destinatario_id = serializers.PrimaryKeyRelatedField(
    queryset=Cliente.objects.ativos(),
    write_only=True, source='destinatario'
)
ruta_id = serializers.PrimaryKeyRelatedField(
    queryset=Ruta.objects.ativas(),
    write_only=True, source='ruta'
)

```

## 5.2.2 FBV con @api\_view

El decorador `@api_view` es un componente fundamental de Django REST Framework (DRF) que se utiliza para convertir las vistas basadas en funciones (FBV) estándar de Django en vistas de API RESTful.

### Funciones clave de @api\_view

- Cuando se envuelve una función con este decorador, este se encarga automáticamente de varias tareas esenciales de la API:
- Filtrado de métodos: garantiza que la vista solo responda a los métodos HTTP especificados (por ejemplo, GET, POST) y devuelva un «405 Method Not Allowed» para los demás.
- Análisis de la solicitud: sustituye el objeto `HttpRequest` estándar de Django por un objeto `Request` de DRF, que cuenta con un atributo `.data` más flexible para gestionar datos JSON o de formularios.
- Representación de la respuesta: permite devolver un objeto `Response` de DRF, que realiza automáticamente la negociación de contenido para devolver el formato correcto

(como JSON) en función de la solicitud del cliente. Gestión de errores: captura las excepciones y las convierte en respuestas de error adecuadas y compatibles con la API.

Python

```
# envios/api_views.py
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from rest_framework import status
from django.shortcuts import get_object_or_404
from .models import Encomienda
from .serializers import EncomiendaSerializer

@api_view(['GET', 'POST'])
@permission_classes([IsAuthenticated])
def encomienda_list(request):
    if request.method == 'GET':
        qs = Encomienda.objects.con_relaciones()
        serializer = EncomiendaSerializer(
            qs, many=True, context={'request': request}
        )
        return Response(serializer.data)

    elif request.method == 'POST':
        serializer = EncomiendaSerializer(
            data=request.data, context={'request': request}
        )
        if serializer.is_valid():
            serializer.save(
                empleado_registro=request.user.empleado
            )
            return Response(
                serializer.data,
                status=status.HTTP_201_CREATED
            )
        return Response(
            serializer.errors,
            status=status.HTTP_400_BAD_REQUEST
        )
```

```

@api_view(['GET', 'PUT', 'PATCH', 'DELETE'])
@permission_classes([IsAuthenticated])
def encomienda_detail(request, pk):
    enc = get_object_or_404(Encomienda, pk=pk)

    if request.method == 'GET':
        return Response(EncomiendaSerializer(enc).data)

    elif request.method in ['PUT', 'PATCH']:
        s = EncomiendaSerializer(
            enc, data=request.data,
            partial=(request.method == 'PATCH')
        )
        if s.is_valid():
            s.save()
            return Response(s.data)
        return Response(s.errors, status=400)

    elif request.method == 'DELETE':
        enc.delete()

    return Response(status=status.HTTP_204_NO_CONTENT)

```

## 5.3. Class Based Views — APIView

La clase `APIView` es el equivalente CBV de `@api_view`. Organiza la lógica en métodos `get()`, `post()`, `put()`, etc. Más estructurado que FBV, más explícito que las vistas genéricas.

En Django REST Framework (DRF), `APIView` es la clase base para todas las vistas basadas en clases (CBV). Se trata de una subclase de la clase `View` estándar de Django, pero está diseñada específicamente para crear API web, ya que proporciona un manejo automatizado de las solicitudes y respuestas, así como la gestión de políticas.

**Diferencias clave con respecto a las vistas estándar de Django** Objetos de solicitud y respuesta:

- Los controladores reciben instancias de `DRFRequest` en lugar de la clase estándar `HttpRequest`, y devuelven objetos `DRFResponse`.

- Gestión de políticas: Gestiona automáticamente la autenticación, los permisos y la limitación de tráfico antes de enviar una solicitud al método del controlador.
- Negociación de contenido: Selecciona automáticamente el renderizador correcto (por ejemplo, JSON o HTML) en función del encabezado Accept de la solicitud.
- Gestión de errores: Captura `APIException` y devuelve respuestas formateadas y adecuadas.

Python

```
# envios/api_views.py
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework.permissions import IsAuthenticated
from rest_framework import status
from django.shortcuts import get_object_or_404
from .models import Encomienda
from .serializers import EncomiendaSerializer, EncomiendaDetailSerializer

class EncomiendaListAPIView(APIView):
    """GET /api/v1/encomiendas/ POST /api/v1/encomiendas/"""
    permission_classes = [IsAuthenticated]

    def get(self, request):
        qs = Encomienda.objects.con_relaciones()
        serializer = EncomiendaSerializer(
            qs, many=True, context={'request': request}
        )
        return Response(serializer.data)

    def post(self, request):
        serializer = EncomiendaSerializer(
            data=request.data, context={'request': request}
        )
        if serializer.is_valid():
            serializer.save(
                empleado_registro=request.user.empleado
            )
            return Response(
                serializer.data, status=status.HTTP_201_CREATED
            )
```

```
    return Response(
        serializer.errors, status=status.HTTP_400_BAD_REQUEST
    )
```

```
class EncomiendaDetailAPIView(APIView):
    """GET/PUT/PATCH/DELETE /api/v1/encomiendas/{pk}/"""
    permission_classes = [IsAuthenticated]

    def get_object(self, pk):
    return get_object_or_404(
        Encomienda.objects.con_relaciones(), pk=pk
    )

    def get(self, request, pk):
    enc = self.get_object(pk)
    return Response(EncomiendaDetailSerializer(enc).data)

    def put(self, request, pk):
    enc = self.get_object(pk)
    s = EncomiendaSerializer(
        enc, data=request.data, context={'request': request}
    )
    if s.is_valid():
        s.save()
        return Response(s.data)
    return Response(s.errors, status=400)

    def patch(self, request, pk):
    enc = self.get_object(pk)
    s = EncomiendaSerializer(
        enc, data=request.data, partial=True,
        context={'request': request}
    )
    if s.is_valid():
        s.save()
        return Response(s.data)
    return Response(s.errors, status=400)

    def delete(self, request, pk):
    enc = self.get_object(pk)
    enc.delete()
    return Response(status=status.HTTP_204_NO_CONTENT)
```

```
# api/urls.py
from django.urls import path
from . import api_views

urlpatterns = [
    path('encomiendas/', api_views.EncomiendaListAPIView.as_view()),
    path('encomiendas/<int:pk>', api_views.EncomiendaDetailAPIView.as_view()),
]
```

## 5.4. Mixins

Los Mixins son clases que aportan métodos concretos. Se combinan con `GenericAPIView` para construir vistas sin repetir código. La ventaja sobre `APIView` es que los métodos CRUD están implementados; solo se mezclan los que se necesitan.

| Mixin              | Método que aporta                                     | Equivale a                |
|--------------------|-------------------------------------------------------|---------------------------|
| ListModelMixin     | <code>list()</code>                                   | GET /recursos/            |
| CreateModelMixin   | <code>create()</code>                                 | POST /recursos/           |
| RetrieveModelMixin | <code>retrieve()</code>                               | GET /recursos/{pk}/       |
| UpdateModelMixin   | <code>update()</code> + <code>partial_update()</code> | PUT/PATCH /recursos/{pk}/ |
| DestroyModelMixin  | <code>destroy()</code>                                | DELETE /recursos/{pk}/    |

Python

```
# envios/api_views.py - Mixins
from rest_framework import mixins, generics
from rest_framework.permissions import IsAuthenticated
from .models import Encomienda
from .serializers import EncomiendaSerializer
```

```

# — List + Create —————
class EncomiendaListCreateView(
    mixins.ListModelMixin,
    mixins.CreateModelMixin,
    generics.GenericAPIView
):
    queryset = Encomienda.objects.con_relaciones()
    serializer_class = EncomiendaSerializer
    permission_classes = [IsAuthenticated]

    def get(self, request, *args, **kwargs):
        return self.list(request, *args, **kwargs)

    def post(self, request, *args, **kwargs):
        return self.create(request, *args, **kwargs)

    def perform_create(self, serializer):
        """Hook: se llama antes de save() en create()"""
        serializer.save(
            empleado_registro=self.request.user.empleado
        )

```

```

# — Retrieve + Update + Destroy —————
class EncomiendaDetailView(
    mixins.RetrieveModelMixin,
    mixins.UpdateModelMixin,
    mixins.DestroyModelMixin,
    generics.GenericAPIView
):
    queryset = Encomienda.objects.con_relaciones()
    serializer_class = EncomiendaSerializer
    permission_classes = [IsAuthenticated]

    def get(self, request, *args, **kwargs):
        return self.retrieve(request, *args, **kwargs)

    def put(self, request, *args, **kwargs):
        return self.update(request, *args, **kwargs)

    def patch(self, request, *args, **kwargs):
        return self.partial_update(request, *args, **kwargs)

```

```
def delete(self, request, *args, **kwargs):

    return self.destroy(request, *args, **kwargs)
```

## 5.5. Vistas Genéricas

Las vistas genéricas son combinaciones predefinidas de Mixins + GenericAPIView. Permiten implementar un CRUD completo con 5 líneas de código. Son la forma recomendada cuando no se necesita lógica personalizada compleja.

| Vista genérica               | Métodos HTTP                     | Descripción          |
|------------------------------|----------------------------------|----------------------|
| ListAPIView                  | GET                              | Solo listar          |
| CreateAPIView                | POST                             | Solo crear           |
| RetrieveAPIView              | GET /{pk}                        | Solo detalle         |
| UpdateAPIView                | PUT/PATCH /{pk}                  | Solo actualizar      |
| DestroyAPIView               | DELETE /{pk}                     | Solo eliminar        |
| ListCreateAPIView            | GET + POST                       | Listar y crear       |
| RetrieveUpdateAPIView        | GET + PUT/PATCH /{pk}            | Detalle y actualizar |
| RetrieveDestroyAPIView       | GET + DELETE /{pk}               | Detalle y eliminar   |
| RetrieveUpdateDestroyAPIView | GET + PUT + PATCH + DELETE /{pk} | Detalle completo     |

Python

```

# envios/api_views.py - Generic Views
from rest_framework import generics
from rest_framework.permissions import IsAuthenticated
from .models import Encomienda
from clientes.models import Cliente
from rutas.models import Ruta
from .serializers import EncomiendaSerializer, EncomiendaDetailSerializer
from .serializers import ClienteSerializer, RutaSerializer

# — Encomiendas: listar + crear —————
class EncomiendaListCreateView(generics.ListCreateAPIView):
    queryset = Encomienda.objects.con_relaciones()
    serializer_class = EncomiendaSerializer
    permission_classes = [IsAuthenticated]

    def perform_create(self, serializer):
        serializer.save(
            empleado_registro=self.request.user.empleado
        )

# — Encomiendas: detalle + actualizar + eliminar —————
class EncomiendaDetailView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Encomienda.objects.con_relaciones()
    permission_classes = [IsAuthenticated]

    def get_serializer_class(self):
        """Usar serializer diferente según el método"""
        if self.request.method == 'GET':
            return EncomiendaDetailSerializer # con objetos anidados
        return EncomiendaSerializer # solo IDs para escritura

# — Clientes: solo lectura —————
class ClienteListView(generics.ListAPIView):
    serializer_class = ClienteSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        return Cliente.objects.ativos()

# — Rutas: solo lectura —————

```

```
class RutaListView(generics.ListAPIView):
    serializer_class = RutaSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        return Ruta.objects.activas()

# api/urls.py
from django.urls import path
from . import api_views

urlpatterns = [
    path('encomiendas/', api_views.EncomiendaListCreateView.as_view()),
    path('encomiendas/<int:pk>/', api_views.EncomiendaDetailView.as_view()),
    path('clientes/', api_views.ClienteListView.as_view()),
    path('rutas/', api_views.RutaListView.as_view()),
]
```

## 6.1. ViewSets y Router

Un ViewSet agrupa en una sola clase todos los endpoints de un recurso. El Router genera automáticamente las URLs del CRUD sin necesidad de escribirlas manualmente.

### Ventajas Principales

- Reducción de Código: Permite agrupar operaciones comunes de CRUD (Crear, Leer, Actualizar, Borrar) en una sola clase, evitando repetir la definición de queryset y serializer\_class en múltiples vistas.
- Configuración de URLs Automática: Al usarse junto con un Router, DRF genera automáticamente todas las rutas necesarias para las acciones del ViewSet.
- Consistencia: Asegura que las convenciones de URL sean uniformes en toda tu API

### Acciones Estándar

A diferencia de las vistas normales, los ViewSets vinculan los métodos HTTP a las siguientes acciones:

| Método HTTP  | Acción         | Descripción                             |
|-------------------------------------------------------------------------------------------------|----------------|-----------------------------------------|
| GET                                                                                             | list           | Obtener una lista de objetos.           |
| POST                                                                                            | create         | Crear un nuevo objeto.                  |
| GET                                                                                             | retrieve       | Obtener un objeto específico por su ID. |
| PUT                                                                                             | update         | Actualizar completamente un objeto.     |
| PATCH                                                                                           | partial_update | Actualizar parcialmente un objeto.      |
| DELETE                                                                                          | destroy        | Eliminar un objeto.                     |

## Tipos de ViewSets

Existen diferentes clases según el nivel de control y funcionalidad requerida:

- **ViewSet**: La clase más básica; no proporciona implementaciones por defecto, por lo que debes definir las acciones manualmente.
- **GenericViewSet**: Hereda de **GenericAPIView**, permitiendo usar mixins para añadir solo las funcionalidades necesarias (ej. solo **list** y **retrieve**).
- **ReadOnlyModelViewSet**: Ideal para APIs de solo lectura; incluye automáticamente las acciones **list** y **retrieve**.
- **ModelViewSet**: La más potente; incluye por defecto todas las operaciones CRUD completas.

## Routers en Django REST Framework (DRF)

En el ecosistema de Django, el término "router" suele referirse a dos conceptos principales dependiendo de si estás trabajando con el núcleo de Django o con extensiones para APIs:

Es el uso más común. Los routers en Django REST Framework automatizan la generación de rutas URL para tus APIs al trabajar con **ViewSets**. En lugar de escribir manualmente cada ruta (GET, POST, etc.), el router las crea siguiendo convenciones estándar.

- **SimpleRouter**: Genera las rutas básicas para operaciones CRUD (list, create, retrieve, update, destroy).
- **DefaultRouter**: Similar al SimpleRouter, pero incluye una vista raíz para la API que muestra todos los enlaces disponibles en formato hipertexto.

### Lab 1

#### ViewSet completo con acciones personalizadas

Implementar `EncomiendaViewSet` con `cambiar_estado`, `con_retraso` y `pendientes`

Python

```
# pos_project/urls.py
```

```

# envios/viewsets.py
from rest_framework import viewsets, status
from rest_framework.decorators import action
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from .models import Encomienda, Empleado
from .serializers import EncomiendaSerializer, EncomiendaDetailSerializer
from config.choices import EstadoEnvio

class EncomiendaViewSet(viewsets.ModelViewSet):
    """
    ModelViewSet genera automáticamente:
    list()      → GET   /encomiendas/
    create()    → POST  /encomiendas/
    retrieve()  → GET   /encomiendas/{pk}/
    update()    → PUT   /encomiendas/{pk}/
    partial_update() → PATCH /encomiendas/{pk}/
    destroy()   → DELETE /encomiendas/{pk}/
    """
    queryset = Encomienda.objects.con_relaciones()
    serializer_class = EncomiendaSerializer
    permission_classes = [IsAuthenticated]

    def get_serializer_class(self):
        if self.action == 'retrieve':
            return EncomiendaDetailSerializer
        return EncomiendaSerializer

    def perform_create(self, serializer):
        serializer.save(
            empleado_registro=self.request.user.empleado
        )

    # — Acción de detalle: POST /encomiendas/{pk}/cambiar_estado/
    @action(detail=True, methods=['post'], url_path='cambiar_estado')
    def cambiar_estado(self, request, pk=None):
        enc = self.get_object()
        nuevo_estado = request.data.get('estado')
        observacion = request.data.get('observacion', '')

        if not nuevo_estado:
            return Response(
                {'error': 'El campo estado es requerido.'},

```

```

        status=status.HTTP_400_BAD_REQUEST
    )
    try:
        empleado = Empleado.objects.get(email=request.user.email)
        enc.cambiar_estado(nuevo_estado, empleado, observacion)
        return Response(EncomiendaSerializer(enc).data)
    except ValueError as e:
        return Response(
            {'error': str(e)},
            status=status.HTTP_400_BAD_REQUEST
        )

# — Acción de lista: GET /encomiendas/con_retraso/
@api_action(detail=False, methods=['get'], url_path='con_retraso')
def con_retraso(self, request):
    qs = Encomienda.objects.con_retraso().con_relaciones()
    return Response(self.get_serializer(qs, many=True).data)

# — Acción de lista: GET /encomiendas/pendientes/
@api_action(detail=False, methods=['get'])
def pendientes(self, request):
    qs = Encomienda.objects.pendientes().con_relaciones()
    return Response(self.get_serializer(qs, many=True).data)

# — Router: registrar el ViewSet
# api/urls.py
from rest_framework.routers import DefaultRouter
from django.urls import path, include
from .viewsets import EncomiendaViewSet
from . import api_views

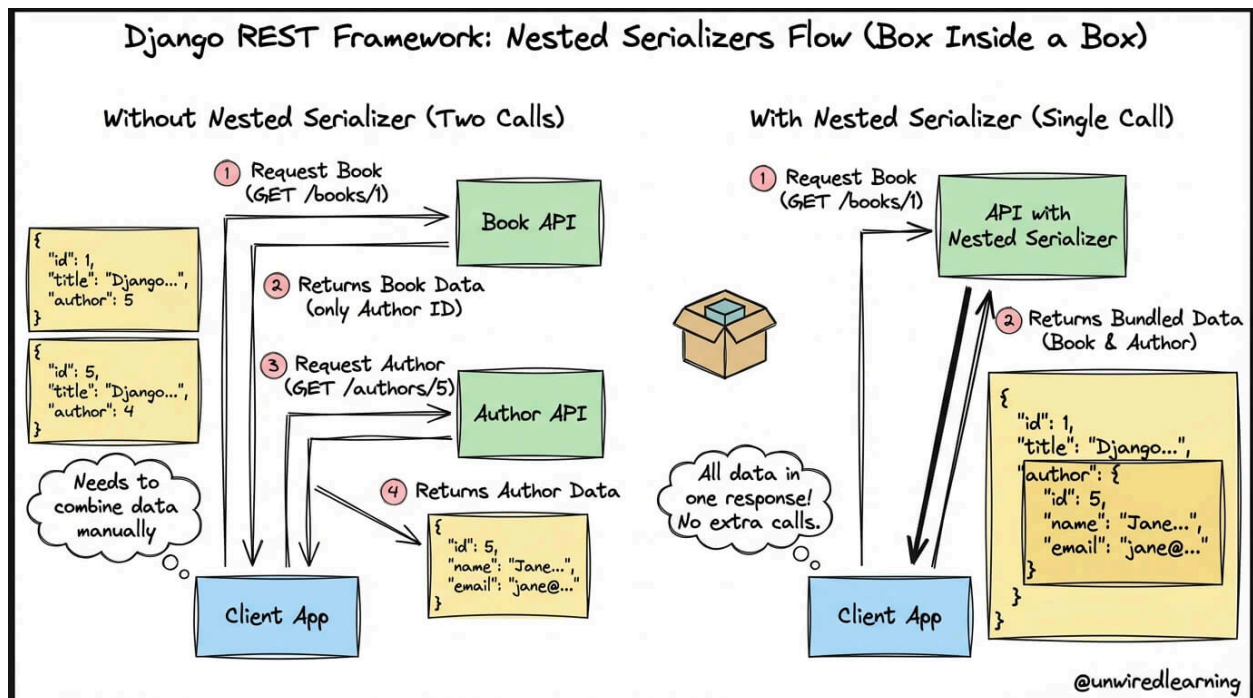
router = DefaultRouter()
router.register('encomiendas', EncomiendaViewSet, basename='encomienda')

urlpatterns = [
    path('', include(router.urls)),
    path('clientes/', api_views.ClienteListView.as_view()),
    path('rutas/', api_views.RutaListView.as_view()),
]

```

## 6.2. Nested Serializers

Un nested serializer (o serializador anidado) en Django REST Framework (DRF) permite representar relaciones entre modelos (como llaves foráneas o ManyToMany) devolviendo el objeto relacionado completo en lugar de solo su ID. En el sistema de encomiendas, el detalle de una encomienda incluye los datos completos del remitente, destinatario y ruta.



Python

# envios/serializers.py - Serializer anidado completo

```
class EncomiendaDetailSerializer(serializers.ModelSerializer):  
    """  
    Para GET: devuelve objetos anidados completos  
    Para POST/PUT/PATCH: acepta solo IDs (write_only)  
    """  
  
    # Campos de solo lectura: objetos anidados completos  
    remitente = ClienteSerializer(read_only=True)
```

```

destinatario = ClienteSerializer(read_only=True)
ruta          = RutaSerializer(read_only=True)

# Campos de solo escritura: aceptar ID para crear/actualizar
remiteente_id = serializers.PrimaryKeyRelatedField(
    queryset=Cliente.objects.ativos(),
    write_only=True, source='remiteente'
)
destinatario_id = serializers.PrimaryKeyRelatedField(
    queryset=Cliente.objects.ativos(),
    write_only=True, source='destinatario'
)
ruta_id = serializers.PrimaryKeyRelatedField(
    queryset=Ruta.objects.ativas(),
    write_only=True, source='ruta'
)

# Historial: los últimos 5 cambios de estado
historial = serializers.SerializerMethodField()

# Propiedades del modelo
esta_entregada = serializers.ReadOnlyField()
tiene_retraso = serializers.ReadOnlyField()
dias_en_transito = serializers.ReadOnlyField()

class Meta:
    model = Encomienda
    fields = [
        'id', 'codigo', 'descripcion', 'peso_kg',
        'remiteente', 'remiteente_id',
        'destinatario', 'destinatario_id',
        'ruta', 'ruta_id',
        'estado', 'costo_envio',
        'fecha_registro', 'fecha_entrega_est', 'fecha_entrega_real',
        'esta_entregada', 'tiene_retraso', 'dias_en_transito',
        'historial', 'observaciones',
    ]

    def get_historial(self, obj):
        """Devuelve los últimos 5 cambios de estado"""
        return HistorialEstadoSerializer(
            obj.historial.all()[:5], many=True
        ).data

```

```
# config/settings.py - ya configurado (recordatorio)
REST_FRAMEWORK = {
    ...
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 15,    # 15 registros por pagina en toda la API
    ...
}

# Con esta configuracion, al hacer GET /api/v1/encomiendas/ la respuesta
# ya viene paginada automaticamente sin agregar nada mas al ViewSet:
# {
#     'count': 120,
#     'next': 'http://localhost:8000/api/v1/encomiendas/?page=2',
#     'previous': null,
#     'results': [{...}, {...}, ...]    <- maximo 15 objetos
# }
```

## 6.3. Paginación

La paginación en Django REST Framework (DRF) permite dividir grandes conjuntos de datos en páginas más pequeñas y manejables, mejorando el rendimiento y la experiencia del usuario. Se implementa configurando clases como `PageNumberPagination` o `CursorPagination` en `settings.py` o directamente en las vistas, utilizando parámetros como `page` o `limit/offset` en la URL

Python

```
# api/pagination.py

from rest_framework.pagination import (
    PageNumberPagination,
    LimitOffsetPagination,
    CursorPagination,
)
```

```

class EncomiendaPagination(PageNumberPagination):
    """
    Paginacion por numero de pagina.
    Uso: GET /api/v1/encomiendas/?page=2
         GET /api/v1/encomiendas/?page=2&page_size=30
    Usada en: EncomiendaViewSet (listado principal)
    """
    page_size = 15 # registros por pagina por defecto
    page_size_query_param = 'page_size' # el cliente puede pedir mas
    max_page_size = 100 # maximo permitido
    page_query_param = 'page' # parametro de la URL

    def get_paginated_response_schema(self, schema):
        """Schema para drf-spectacular (documentacion Swagger)"""
        return {
            'type': 'object',
            'properties': {
                'count': {'type': 'integer', 'example': 120},
                'next': {'type': 'string', 'nullable': True},
                'previous': {'type': 'string', 'nullable': True},
                'results': schema,
            }
        }

class ClientePagination(PageNumberPagination):
    """
    Paginacion para el listado de clientes.
    Uso: GET /api/v1/clientes/?page=2
    Usada en: ClienteListView
    """
    page_size = 20
    page_size_query_param = 'page_size'
    max_page_size = 50

class HistorialPagination(LimitOffsetPagination):
    """
    Paginacion por limit/offset para el historial de una encomienda.
    Uso: GET /api/v1/encomiendas/1/historial/?limit=5&offset=10
    Usada en: accion historial del EncomiendaViewSet
    """
    default_limit = 10
    max_limit = 50

class EncomiendaCursorPagination(CursorPagination):
    """
    Paginacion por cursor. Eficiente para grandes volúmenes de datos.
    Uso: GET /api/v1/encomiendas/feed/?cursor=cD0yMDI2LTA0...
    Usada en: accion feed del EncomiendaViewSet (tiempo real)
    """
    page_size = 15

```

```
ordering = '-fecha_registro'

# Devuelve next/previous como cursores opacos (no expone el total)
```

## Aplicar paginación en el ViewSet — envios/viewsets.py

Abre `envios/viewsets.py` y modifica el `EncomiendaViewSet` para usar los paginadores personalizados. El ViewSet ya debe existir con el código del tema 6.1:

Python

```
# pos_project/urls.py

# envios/viewsets.py
# MODIFICAR: agregar los imports y atributos de paginacion

from rest_framework import viewsets, status
from rest_framework.decorators import action
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from rest_framework.pagination import LimitOffsetPagination

# Importar los paginadores del proyecto
from api.pagination import EncomiendaPagination, HistorialPagination

from .models import Encomienda, Empleado
from .serializers import (
    EncomiendaSerializer,
    EncomiendaDetailSerializer,
    HistorialEstadoSerializer,
)
from config.choices import EstadoEnvio

class EncomiendaViewSet(viewsets.ModelViewSet):
    queryset = Encomienda.objects.con_relaciones()
    serializer_class = EncomiendaSerializer
    permission_classes = [IsAuthenticated]
    pagination_class = EncomiendaPagination # <- paginador del proyecto

    def get_serializer_class(self):
        if self.action == 'retrieve':
            return EncomiendaDetailSerializer
        return EncomiendaSerializer

    def perform_create(self, serializer):
```

```

        serializer.save(
            empleado_registro=self.request.user.empleado
        )

# — Acciones ya implementadas en tema 6.1 —————
@action(detail=True, methods=['post'], url_path='cambiar_estado')
def cambiar_estado(self, request, pk=None):
    enc = self.get_object()
    nuevo_estado = request.data.get('estado')
    observacion = request.data.get('observacion', '')
    if not nuevo_estado:
        return Response({'error': 'El campo estado es requerido.'},
            status=400)
    try:
        empleado = Empleado.objects.get(email=request.user.email)
        enc.cambiar_estado(nuevo_estado, empleado, observacion)
        return Response(EncomiendaSerializer(enc).data)
    except ValueError as e:
        return Response({'error': str(e)}, status=400)

@action(detail=False, methods=['get'], url_path='con_retraso')
def con_retraso(self, request):
    qs = Encomienda.objects.con_retraso().con_relaciones()
    return Response(self.get_serializer(qs, many=True).data)

@action(detail=False, methods=['get'])
def pendientes(self, request):
    qs = Encomienda.objects.pendientes().con_relaciones()
    return Response(self.get_serializer(qs, many=True).data)

# — NUEVA accion: historial con paginacion propia —————
@action(detail=True, methods=['get'], url_path='historial')
def historial(self, request, pk=None):
    """
    GET /api/v1/encomiendas/{pk}/historial/
    GET /api/v1/encomiendas/{pk}/historial/?limit=5&offset=10
    Devuelve el historial de cambios de estado paginado.
    """
    enc = self.get_object()
    qs = enc.historial.select_related('empleado').order_by('-fecha_cambio')
    # Usar el paginador especifico para historial (LimitOffset)
    paginator = HistorialPagination()
    page = paginator.paginate_queryset(qs, request)
    if page is not None:
        serializer = HistorialEstadoSerializer(page, many=True)
        return paginator.get_paginated_response(serializer.data)
    serializer = HistorialEstadoSerializer(qs, many=True)
    return Response(serializer.data)

# — NUEVA accion: estadisticas (sin paginar, datos resumidos) —
@action(detail=False, methods=['get'])
def estadisticas(self, request):
    """
    GET /api/v1/encomiendas/estadisticas/
    Devuelve contadores globales (no requiere paginacion).
    """

```

```

"""
from django.utils import timezone
hoy = timezone.now().date()
return Response({
    'total_activas': Encomienda.objects.activas().count(),
    'en_transito': Encomienda.objects.en_transito().count(),
    'con_retraso': Encomienda.objects.con_retraso().count(),
    'entregadas_hoy': Encomienda.objects.filter(
        estado='EN',
        fecha_entrega_real=hoy
    ).count(),
})

```

## Aplicar paginación en las vistas genéricas — envios/api\_views.py

Abre `envios/api_views.py` y actualiza las vistas genéricas con sus paginadores correspondientes:

Python

```

# envios/api_views.py
# MODIFICAR: agregar pagination_class a cada vista

from rest_framework import generics
from rest_framework.permissions import IsAuthenticated
from clientes.models import Cliente
from rutas.models import Ruta
from .serializers import ClienteSerializer, RutaSerializer

# Importar los paginadores del proyecto
from api.pagination import ClientePagination

class ClienteListView(generics.ListAPIView):
    serializer_class = ClienteSerializer
    permission_classes = [IsAuthenticated]
    pagination_class = ClientePagination # 20 por pagina

    def get_queryset(self):
        return Cliente.objects.activos()

class RutaListView(generics.ListAPIView):
    serializer_class = RutaSerializer
    permission_classes = [IsAuthenticated]

```

```
# Las rutas son pocas: no paginamos (None deshabilita la paginacion)
pagination_class = None

def get_queryset(self):
    return Ruta.objects.activas()
```

## Formato de respuesta paginada

Así se ve la respuesta JSON cuando el endpoint tiene paginación activa:

Python

```
# GET http://localhost:8000/api/v1/encomiendas/
# (o cualquier pagina: ?page=2 | ?page=2&page_size=5)

{
  "count": 120,
  "next": "http://localhost:8000/api/v1/encomiendas/?page=2",
  "previous": null,
  "results": [
    {
      "id": 45,
      "codigo": "ENC-2026-045",
      "estado": "TR",
      "estado_display": "En transito",
      "tiene_retraso": false,
      ...
    },
    ...
  ]
}

# GET http://localhost:8000/api/v1/encomiendas/1/historial/?limit=5&offset=10
# (LimitOffset para el historial)

{
  "count": 18,
  "next":
"http://localhost:8000/api/v1/encomiendas/1/historial/?limit=5&offset=15",
  "previous":
"http://localhost:8000/api/v1/encomiendas/1/historial/?limit=5&offset=5",
  "results": [
    {
      "estado_anterior": "PE",
      "estado_anterior_display": "Pendiente",
      "estado_nuevo": "TR",

```

```

        "estado_nuevo_display": "En transito",
        "empleado_nombre": "Mendoza Cruz, Luis",
        "fecha_cambio": "2026-04-18T10:30:00Z"
    },
    ...
]
}

# GET http://localhost:8000/api/v1/clientes/
# (ClientePagination: 20 por pagina)

{
    "count": 87,
    "next": "http://localhost:8000/api/v1/clientes/?page=2",
    "previous": null,
    "results": [...]
}

# GET http://localhost:8000/api/v1/rutas/
# (sin paginacion: pagination_class = None)

[
    {"codigo": "LIM-TRU", "origen": "Lima", "destino": "Trujillo", ...},
    {"codigo": "LIM-AQP", "origen": "Lima", "destino": "Arequipa", ...}
]

```

#### RESUMEN — ARCHIVOS MODIFICADOS EN ESTA SECCIÓN

|                     |                                                                                                                                                                |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| api/pagination.py   | <- IMPLEMENTADO: EncomiendaPagination (page/page_size),<br>ClientePagination (20/pagina),<br>HistorialPagination (limit/offset),<br>EncomiendaCursorPagination |
| envios/viewsets.py  | <- MODIFICADO: pagination_class = EncomiendaPagination,<br>nueva accion historial() con HistorialPagination,<br>nueva accion estadisticas() sin paginacion     |
| envios/api_views.py | <- MODIFICADO: ClienteListView usa ClientePagination,<br>RutaListView usa pagination_class = None                                                              |
| config/settings.py  | <- sin cambios (ya tenia la config global del Paso 4)                                                                                                          |

## 6.4. Filtering

Los filtros en Django REST Framework (DRF) permiten refinar conjuntos de datos (QuerySets) mediante parámetros en la URL, mejorando la eficiencia y experiencia de usuario. Se implementan usando `filter_backends` en las vistas, permitiendo filtrado básico, búsqueda por palabras clave (`SearchFilter`), ordenamiento (`OrderingFilter`) o filtros avanzados con `django-filter`.

Principales formas de filtrar en DRF:

- **Filtrado Básico con `DjangoFilterBackend`:**

Es el método recomendado para filtros exactos o de rango. Requiere instalar `django-filter` y añadirlo a `INSTALLED_APPS`.

python

```
# views.py
from django_filters.rest_framework import DjangoFilterBackend
from rest_framework import generics
from .models import Producto
from .serializers import ProductoSerializer

class ProductListView(generics.ListAPIView):
    queryset = Producto.objects.all()
    serializer_class = ProductoSerializer
    filter_backends = [DjangoFilterBackend]
    filterset_fields = ['categoria', 'estado'] # Campos a filtrar
```

URL ejemplo: `/api/productos/?categoria=electronica&estado=activo`

- **Búsqueda (`SearchFilter`):**

Permite buscar términos dentro de campos de texto específicos.

python

```
from rest_framework import filters
class ProductListView(generics.ListAPIView):
    # ...
    filter_backends = [filters.SearchFilter]
    search_fields = ['^nombre', 'descripcion'] # '^' inicia con, '=' es
```

- **Ordenamiento (OrderingFilter):**

Permite ordenar los resultados según parámetros en la URL.

python

```
from rest_framework import filters
class ProductListView(generics.ListAPIView):
    # ...
    filter_backends = [filters.OrderingFilter]
    ordering_fields = ['precio', 'fecha_creacion']
```

URL ejemplo: /api/productos/?ordering=-precio (descendente)

Vamos a implementarlo

Primero debemos instalar el paquete django-filter: **pip install django-filter**

Luego lo agregamos en [settings.py](#) en la parte de aplicaciones instaladas

```
Python
INSTALLED_APPS += ['django_filters']
```

Luego creamos una clase llamada filters.py

Python

```
# api/filters.py
from django_filters.rest_framework import (
    FilterSet, CharFilter, ChoiceFilter, BooleanFilter
)

# api/filters.py
from django_filters.rest_framework import (
    FilterSet, CharFilter, ChoiceFilter, BooleanFilter
)
from django.utils import timezone
from .models import Encomienda
from config.choices import EstadoEnvio

class EncomiendaFilter(FilterSet):
    estado = ChoiceFilter(choices=EstadoEnvio.choices)
    ruta = CharFilter(field_name='ruta__codigo', lookup_expr='iexact')
    remitente = CharFilter(field_name='remitente__nro_doc')
    desde = CharFilter(field_name='fecha_registro__date', lookup_expr='gte')
    hasta = CharFilter(field_name='fecha_registro__date', lookup_expr='lte')
    con_retraso = CharFilter(method='filter_retraso')

    def filter_retraso(self, queryset, name, value):
        if value.lower() == 'true':
            return queryset.con_retraso()
        return queryset

    class Meta:
        model = Encomienda
        fields = ['estado', 'ruta', 'remitente', 'desde', 'hasta']

# envios/viewsets.py
from rest_framework.filters import SearchFilter, OrderingFilter
from django_filters.rest_framework import DjangoFilterBackend
from api.filters import EncomiendaFilter

class EncomiendaViewSet(viewsets.ModelViewSet):
    filter_backends = [DjangoFilterBackend, SearchFilter, OrderingFilter]
    filterset_class = EncomiendaFilter
    search_fields = [
```

```
'codigo',
'remitente__apellidos',
'destinatario__apellidos',
'descripcion',
]
ordering_fields = ['fecha_registro', 'peso_kg', 'costo_envio']
ordering        = ['-fecha_registro'] # orden por defecto

# Ejemplos de uso:
# GET /api/v1/encomiendas/?estado=TR
# GET /api/v1/encomiendas/?search=Lima
# GET /api/v1/encomiendas/?ordering=-peso_kg
# GET /api/v1/encomiendas/?desde=2026-01-01&hasta=2026-04-30
# GET /api/v1/encomiendas/?estado=TR&ordering=-fecha_registro&page=2

# GET /api/v1/encomiendas/?con_retraso=true
```

## 6.5. Seguridad y Autenticación

DRF soporta varios esquemas de autenticación. Para el sistema de encomiendas usaremos JWT (JSON Web Tokens) con SimpleJWT, el estándar actual para APIs REST.

### 6.5.1 JWT con django-rest-framework-simplejwt

Primero instalamos el paquete necesario:

```
Python
pip install django-rest-framework-simplejwt
```

```
Python

# config/settings.py
from datetime import timedelta

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
}

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
    'ROTATE_REFRESH_TOKENS': True,
    'BLACKLIST_AFTER_ROTATION': True,
    'AUTH_HEADER_TYPES': ('Bearer',),
    'USER_ID_FIELD': 'id',
```

```
'USER_ID_CLAIM':      'user_id',  
}
```

Python

```
# config/urls.py  
from rest_framework_simplejwt.views import (  
    TokenObtainPairView,  
    TokenRefreshView,  
    TokenBlacklistView,  
)  
  
urlpatterns += [  
    path('api/v1/auth/token/',  
        TokenObtainPairView.as_view(), name='token_obtain'),  
    path('api/v1/auth/token/refresh/',  
        TokenRefreshView.as_view(), name='token_refresh'),  
    path('api/v1/auth/token/blacklist/',  
        TokenBlacklistView.as_view(), name='token_blacklist'),  
]
```

## JWT personalizado con datos del empleado

Python

```
# JWT personalizado con datos del empleado  
from rest_framework_simplejwt.serializers import TokenObtainPairSerializer  
from rest_framework_simplejwt.views import TokenObtainPairView  
  
class EncomiendaTokenSerializer(TokenObtainPairSerializer):  
    @classmethod  
    def get_token(cls, user):  
        token = super().get_token(user)
```

```
class EncomiendaTokenView(TokenObtainPairView):
    serializer_class = EncomiendaTokenSerializer
```

```
# config/urls.py
```

```
# — Flujo de autenticación —————  
# 1. Obtener tokens:  
#     POST /api/v1/auth/token/  
#     Body: {"username": "juan", "password": "secret"}  
#     Respuesta: {"access": "eyJhbGciOiJIUzI1NiIsInR5cGE6bnVlc3kiLCJ0eXA6ImFpdCI6eyJuYW5hZyI6ImNyeXBkaW8ifX0=", "refresh": "eyJhbGciOiJIUzI1NiIsInR5cGE6bnVlc3kiLCJ0eXA6ImFpdCI6eyJuYW5hZyI6ImNyeXBkaW8ifX0="}  
  
# 2. Usar el token en cada request:  
#     GET /api/v1/encomiendas/  
#     Header: Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cGE6bnVlc3kiLCJ0eXA6ImFpdCI6eyJuYW5hZyI6ImNyeXBkaW8ifX0=  
  
# 3. Renovar cuando expira (sin login de nuevo):  
#     POST /api/v1/auth/token/refresh/  
#     Body: {"refresh": "eyJhbGciOiJIUzI1NiIsInR5cGE6bnVlc3kiLCJ0eXA6ImFpdCI6eyJuYW5hZyI6ImNyeXBkaW8ifX0="}
```

```
# Respuesta: {"access": "eyJhbGciOiJIUzI1NiIsInR5cGE6Ijoi"}

```

## 6.5.2 Permisos personalizados

Python

```
# api/permissions.py
from rest_framework.permissions import BasePermission
from envios.models import Empleado

class EsEmpleadoActivo(BasePermission):
    """Solo empleados activos del sistema pueden acceder"""
    message = 'Solo empleados activos tienen acceso a esta API.'

    def has_permission(self, request, view):
        if not request.user.is_authenticated:
            return False
        return Empleado.objects.filter(
            email=request.user.email, estado=1
        ).exists()

class EsPropietarioOAdmin(BasePermission):
    """El usuario puede ver/editar solo sus propias encomiendas, a menos que sea
    admin"""
    def has_object_permission(self, request, view, obj):
        if request.user.is_staff:
            return True
        return obj.empleado_registro.email == request.user.email

# Uso en el ViewSet
from api.permissions import EsEmpleadoActivo, EsPropietarioOAdmin

class EncomiendaViewSet(viewsets.ModelViewSet):

```

```

permission_classes = [EsEmpleadoActivo]

def get_permissions(self):
    """Permisos distintos según la acción"""
    if self.action in ['update', 'partial_update', 'destroy']:
        return [EsEmpleadoActivo(), EsPropietarioOAdmin()]

    return [EsEmpleadoActivo()]

```

### 6.5.3 HttpOnly Cookies

Las cookies HttpOnly son una función de seguridad que impide que los scripts del lado del cliente (como JavaScript) accedan a datos confidenciales almacenados en las cookies. Al activar el indicador HttpOnly, se indica a los navegadores que no expongan estas cookies a ``document.cookie``, lo que reduce el riesgo de secuestro de sesión mediante ataques de tipo «Cross-Site Scripting» (XSS).

#### Características principales:

- Inaccesibles para JavaScript: JavaScript no puede leerlas ni modificarlas, lo que impide de forma efectiva el robo a través de XSS.
- Solo del lado del servidor: Están diseñadas para que solo el servidor backend pueda configurarlas y leerlas.
- Transmisión automática: El navegador incluye automáticamente estas cookies en las solicitudes HTTP, lo que garantiza su funcionamiento para la autenticación sin necesidad de acceso a JavaScript.
- Uso principal: Ideales para almacenar tokens de sesión confidenciales (por ejemplo, ID de sesión).

Python

```

# envios/api_auth.py
from rest_framework.views import APIView
from rest_framework.response import Response

```

```

from rest_framework_simplejwt.tokens import RefreshToken
from django.contrib.auth import authenticate

class LoginCookieView(APIView):
    permission_classes = []

    def post(self, request):
        username = request.data.get('username')
        password = request.data.get('password')
        user = authenticate(username=username, password=password)

        if not user:
            return Response(
                {'error': 'Credenciales inválidas.'},
                status=401
            )

        refresh = RefreshToken.for_user(user)
        response = Response({'message': 'Login exitoso.'})

        # Guardar el JWT en una HttpOnly Cookie
        response.set_cookie(
            key='access_token',
            value=str(refresh.access_token),
            httponly=True, # no accesible desde JS
            secure=True, # solo por HTTPS
            samesite='Lax', # protege contra CSRF
            max_age=3600, # 1 hora
        )
        response.set_cookie(
            key='refresh_token',
            value=str(refresh),
            httponly=True,
            secure=True,
            samesite='Lax',
            max_age=604800, # 7 días
        )
        return response

class LogoutCookieView(APIView):
    def post(self, request):
        response = Response({'message': 'Logout exitoso.'})
        response.delete_cookie('access_token')
        response.delete_cookie('refresh_token')

```

```
return response
```

## 6.6. Validaciones en Serializers

DRF ofrece tres niveles de validación en los serializers: validators de campo, validate\_{campo} para un campo específico, y validate() para reglas cruzadas entre campos.

| Nivel                  | Método              | Cuándo usarlo                         |
|------------------------|---------------------|---------------------------------------|
| Validators en el campo | validators=[...]    | Reglas simples y reutilizables        |
| Validación de campo    | validate_{nombre}() | Regla sobre un campo específico       |
| Validación cruzada     | validate()          | Reglas que involucran más de un campo |

Python

```
# envios/serializers.py
from rest_framework import serializers
from django.utils import timezone

class EncomiendaSerializer(serializers.ModelSerializer):

    # — Validator de campo: validate_{nombre} —————
    def validate_peso_kg(self, value):
        if value <= 0:
            raise serializers.ValidationError(
                'El peso debe ser mayor a 0 kg.'
            )
        if value > 500:
            raise serializers.ValidationError(
                'El peso máximo permitido es 500 kg.'
            )
        return value
```

```

def validate_codigo(self, value):
    if not value.startswith('ENC-'):
        raise serializers.ValidationError(
            'El código debe comenzar con ENC-'
        )
    return value.upper()

def validate_costo_envio(self, value):
    if value < 0:
        raise serializers.ValidationError(
            'El costo no puede ser negativo.'
        )
    return value

# — Validación cruzada: validate() —————
def validate(self, data):
    errors = {}

    # Regla 1: remitente != destinatario
    if data.get('remitente') == data.get('destinatario'):
        errors['destinatario'] = (
            'El destinatario no puede ser el mismo que el remitente.'
        )

    # Regla 2: fecha estimada no en el pasado
    fecha_est = data.get('fecha_entrega_est')
    if fecha_est and fecha_est < timezone.now().date():
        errors['fecha_entrega_est'] = (
            'La fecha estimada no puede ser en el pasado.'
        )

    # Regla 3: costo mínimo según la ruta
    ruta = data.get('ruta')
    costo = data.get('costo_envio')
    if ruta and costo and costo < float(ruta.precio_base):
        errors['costo_envio'] = (
            f'El costo mínimo para esta ruta es S/ {ruta.precio_base}.'
        )

    if errors:
        raise serializers.ValidationError(errors)
    return data

# Respuesta de error (status 400):

```

```
# {  
#   'destinatario': ['El destinatario no puede ser el mismo...'],  
#   'fecha_entrega_est': ['La fecha estimada no puede ser...']  
# }
```

## 6.7. Documentación de la API

drf-spectacular genera documentación **OpenAPI 3.0** automáticamente leyendo el código de los serializers, viewsets y permisos. El resultado es una interfaz Swagger UI interactiva donde se puede explorar y probar cada endpoint del sistema de encomiendas sin necesidad de Postman u otra herramienta externa.

### ESTADO ACTUAL DEL PROYECTO

drf-spectacular ya está instalado y configurado desde el Paso 1 de la introducción. En esta sección verificamos que funciona correctamente, aprendemos a navegar la interfaz Swagger y anotamos los endpoints personalizados con `@extend_schema`.

Si pasaste directamente a esta sección, confirma que en `config/settings.py` tienes:

```
REST_FRAMEWORK['DEFAULT_SCHEMA_CLASS'] = 'drf_spectacular.openapi.AutoSchema'  
SPECTACULAR_SETTINGS = {'TITLE': 'API Sistema de Encomiendas', ...}
```

Y en `config/urls.py` las rutas:

```
path('api/schema/', SpectacularAPIView.as_view())  
path('api/docs/', SpectacularSwaggerView.as_view())
```

## Paso 1 — Verificar que drf-spectacular está instalado

Python

```
# Confirmar la instalación dentro del contenedor
docker compose exec web pip show drf-spectacular
# Resultado esperado:
# Name: drf-spectacular
# Version: 0.27.2

# Si no está instalado, agregar al requirements.txt y reconstruir:
# drf-spectacular==0.27.2

# docker compose build && docker compose up -d
```

## Paso 2 — Confirmar la configuración en settings.py

Verifica que estos bloques estén presentes en `config/settings.py`. Ya deben existir desde la instalación inicial:

Python

```
# config/settings.py

INSTALLED_APPS = [
    ...
    'drf_spectacular',  # <- debe estar en INSTALLED_APPS
    ...
]

REST_FRAMEWORK = {
    ...
    # Esta línea hace que drf-spectacular lea todos los ViewSets
    'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',
}

# Metadatos que aparecen en la cabecera de Swagger UI
SPECTACULAR_SETTINGS = {
    'TITLE': 'API Sistema de Gestión de Encomiendas',
    'DESCRIPTION': '''
API REST para gestionar el ciclo de vida de encomiendas.
Incluye registro de envíos, cambio de estado, historial
'''
}
```

```

y estadísticas del sistema.
'''
'VeRSION': '1.0.0',
'SERVE_INCLUDE_SCHEMA': False, # no mostrar el schema en Swagger
'COMPONENT_SPLIT_REQUEST': True, # esquemas separados para request y
response
'SORT_OPERATIONS': False, # mantener el orden del router
'TAGS': [
{'name': 'Encomiendas', 'description': 'Gestión de envíos'},
{'name': 'Clientes', 'description': 'Listado de clientes activos'},
{'name': 'Rutas', 'description': 'Rutas disponibles'},
{'name': 'Auth', 'description': 'Autenticación JWT'},
],
}

```

## Paso 3 — Confirmar las URLs en config/urls.py

Estas URLs ya deben existir desde la instalación. Verifica que estén en `config/urls.py`:

```

Python
# config/urls.py
from drf_spectacular.views import (
    SpectacularAPIView, # endpoint que sirve el schema YAML/JSON
    SpectacularSwaggerView, # interfaz Swagger UI (interactiva)
    SpectacularRedocView, # interfaz ReDoc (solo lectura, más limpia)
)

urlpatterns = [
    ...
    # Schema en formato OpenAPI 3.0 (YAML o JSON)
    path('api/schema/', SpectacularAPIView.as_view(), name='schema'),

    # Swagger UI: interfaz interactiva para probar la API
    path('api/docs/', SpectacularSwaggerView.as_view(url_name='schema'),
    name='swagger'),

    # ReDoc: documentación de solo lectura más limpia
    path('api/redoc/', SpectacularRedocView.as_view(url_name='schema'),
    name='redoc'),
    ...
]

```

```
]
```

## Paso 4 — Generar y verificar el schema

Antes de abrir el navegador, verifica que drf-spectacular puede leer toda la API sin errores:

```
None
# Generar el schema y verificar que no hay errores
docker compose exec web python manage.py spectacular --validate
# Resultado esperado: 'Schema validated successfully!'

# Si hay advertencias las muestra pero no falla
# Los errores reales impiden que Swagger funcione correctamente

# Tambien puedes exportar el schema a un archivo YAML
docker compose exec web python manage.py spectacular --file schema.yml

# Ver el contenido del schema generado
docker compose exec web cat schema.yml | head -50

# Debe mostrar el encabezado OpenAPI 3.0 con los paths de la API
```

## Paso 5 — Explorar Swagger UI en el navegador

Con los contenedores corriendo, abre el navegador y navega a las siguientes URLs:

```
None

# Swagger UI (recomendada para desarrollo y pruebas)
http://localhost:8000/api/docs/

# ReDoc (recomendada para documentar a clientes externos)

# Swagger UI (recomendada para desarrollo y pruebas)
```

```
http://localhost:8000/api/docs/

# ReDoc (recomendada para documentar a clientes externos)
http://localhost:8000/api/redoc/

# Schema raw en formato YAML
http://localhost:8000/api/schema/

# Schema raw en formato JSON
http://localhost:8000/api/schema/?format=json

# En Swagger UI debes ver:
# - El titulo: 'API Sistema de Gestión de Encomiendas'
# - La version: 1.0.0
# - Los grupos de endpoints por tag: Encomiendas, Clientes, Rutas, Auth
# - Cada endpoint con su metodo HTTP, URL y descripción
```

## Paso 6 — Autenticarse en Swagger UI para probar los endpoints

Todos los endpoints requieren autenticación JWT. Swagger UI tiene un botón para ingresar el token. Sigue estos pasos:

```
Python

# Paso A: obtener el token desde la terminal
curl -X POST http://localhost:8000/api/v1/auth/token/ \
  -H 'Content-Type: application/json' \
  -d '{"username": "tu_superuser", "password": "tu_password"}'

# Respuesta:
# {
#   "access":
#     "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2t1b190eXB1IjoiaWVWNjZlZmVzaC4uLn0",
#   "refresh":
#     "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2t1b190eXB1IjoicmVmcVzaC4uLn0"
# }

# Paso B: copiar el valor del campo 'access' (sin comillas)
```

```
# Paso C: en Swagger UI hacer clic en el boton 'Authorize' (candado)
# ubicado en la parte superior derecha de la pagina

# Paso D: en el campo 'bearerAuth (http, Bearer)'
# escribir: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
# (la palabra Bearer seguida de un espacio y el token access)

# Paso E: hacer clic en 'Authorize' y luego 'Close'

# A partir de aqui, todas las pruebas desde Swagger incluyen
# automaticamente el token en la cabecera Authorization
```

## Paso 7 — Probar endpoints desde Swagger UI

Con el token activo, prueba cada endpoint directamente desde el navegador:

Python

```
# — Probar GET /api/v1/encomiendas/ —————
# 1. Hacer clic en 'GET /api/v1/encomiendas/'
# 2. Hacer clic en 'Try it out'
# 3. Hacer clic en 'Execute'
# 4. Ver la respuesta en 'Response body':
# {
#   'count': 50,
#   'next': 'http://localhost:8000/api/v1/encomiendas/?page=2',
#   'results': [...]
# }

# — Probar POST /api/v1/encomiendas/ —————
# 1. Hacer clic en 'POST /api/v1/encomiendas/'
# 2. Hacer clic en 'Try it out'
# 3. Reemplazar el body con datos reales:
# {
#   "codigo":      "ENC-2026-TEST",
#   "descripcion": "Documentos legales",
#   "peso_kg":     "1.50",
#   "remitente_id": 1,
```

```

    "destinatario_id": 2,
    "ruta_id": 1,
    "costo_envio": "25.00",
    "fecha_entrega_est": "2026-05-30"
}
# 4. Hacer clic en 'Execute'
# 5. Ver la respuesta 201 Created con la encomienda creada

# — Probar POST /api/v1/encomiendas/{id}/cambiar_estado/ —
# 1. Hacer clic en la accion 'cambiar_estado'
# 2. Hacer clic en 'Try it out'
# 3. Ingresar el id de la encomienda
# 4. En el body:
{
    "estado": "TR",
    "observacion": "Recogido en agencia Lima"
}

# 5. Execute -> ver la respuesta 200 con el estado actualizado

```

## Paso 8 — Anotar el ViewSet con @extend\_schema

Por defecto drf-spectacular genera la documentación automáticamente, pero las acciones personalizadas (@action) necesitan anotación manual para tener una descripción precisa en Swagger. Abre envios/viewsets.py y agrega los decoradores:

```

Python
# envios/viewsets.py
from drf_spectacular.utils import (
    extend_schema,
    extend_schema_view,
    OpenApiParameter,
    OpenApiResponse,
    OpenApiExample,
)
from drf_spectacular.types import OpenApiTypes

```

```

# Anotar todas las acciones estandar del ViewSet de una vez
@extend_schema_view(
    list=extend_schema(
        summary='Listar encomiendas',
        description='Devuelve la lista paginada de encomiendas. Soporta filtros por estado, busqueda y ordenamiento.',
        tags=['Encomiendas'],
    ),
    create=extend_schema(
        summary='Crear encomienda',
        description='Registra una nueva encomienda en el sistema.',
        tags=['Encomiendas'],
    ),
    retrieve=extend_schema(
        summary='Detalle de encomienda',
        description='Devuelve los datos completos de una encomienda con remitente, destinatario, ruta e historial de estados.',
        tags=['Encomiendas'],
    ),
    update=extend_schema(summary='Actualizar encomienda',
tags=['Encomiendas']),
    partial_update=extend_schema(summary='Actualizar parcial',
tags=['Encomiendas']),
    destroy=extend_schema(summary='Eliminar encomienda',
tags=['Encomiendas']),
)
class EncomiendaViewSet(viewsets.ModelViewSet):
    queryset = Encomienda.objects.con_relaciones()
    permission_classes = [IsAuthenticated]
    pagination_class = EncomiendaPagination

    # ... get_serializer_class, perform_create, get_queryset ...

    @extend_schema(
        summary='Cambiar estado de encomienda',
        description=''
        Cambia el estado de una encomienda y registra el cambio
        automaticamente en el historial de estados.
        Estados disponibles:
        - PE: Pendiente
        - TR: En transito
        - DE: En destino
    )

```

```

        - EN: Entregado
        - DV: Devuelto
    '''
    request=OpenApiTypes.OBJECT,
    responses={
        200: EncomiendaSerializer,
        400: OpenApiResponse(description='Estado invalido o ya en ese
estado'),
    },
    examples=[
        OpenApiExample(
            'Pasar a En transito',
            value={'estado': 'TR', 'observacion': 'Recogido en agencia Lima'},
            request_only=True,
        ),
        OpenApiExample(
            'Marcar como Entregado',
            value={'estado': 'EN', 'observacion': 'Entregado al destinatario'},
            request_only=True,
        ),
    ],
    tags=['Encomiendas'],
)
@api_action(detail=True, methods=['post'], url_path='cambiar_estado')
def cambiar_estado(self, request, pk=None):
    enc = self.get_object()
    nuevo_estado = request.data.get('estado')
    observacion = request.data.get('observacion', '')
    if not nuevo_estado:
        return Response({'error': 'El campo estado es requerido.'},
status=400)
    try:
        empleado = Empleado.objects.get(email=request.user.email)
        enc.cambiar_estado(nuevo_estado, empleado, observacion)
        return Response(EncomiendaSerializer(enc).data)
    except ValueError as e:
        return Response({'error': str(e)}, status=400)

@api_extend_schema(
    summary='Encomiendas con retraso',
    description='Lista todas las encomiendas activas cuya fecha estimada de
entrega ya pasó.',
    tags=['Encomiendas'],
    responses={200: EncomiendaSerializer(many=True)},

```

```

    )
    @action(detail=False, methods=['get'], url_path='con_retraso')
    def con_retraso(self, request):
        qs = Encomienda.objects.con_retraso().con_relaciones()
        return Response(self.get_serializer(qs, many=True).data)

    @extend_schema(
        summary='Encomiendas pendientes',
        description='Lista todas las encomiendas en estado Pendiente.',
        tags=['Encomiendas'],
    )
    @action(detail=False, methods=['get'])
    def pendientes(self, request):
        qs = Encomienda.objects.pendientes().con_relaciones()
        return Response(self.get_serializer(qs, many=True).data)

    @extend_schema(
        summary='Historial de estados',
        description='Devuelve el historial de cambios de estado de una encomienda, paginado con limit/offset.',
        parameters=[
            OpenApiParameter('limit', type=int, description='Numero de resultados', default=10),
            OpenApiParameter('offset', type=int, description='Posicion de inicio', default=0),
        ],
        tags=['Encomiendas'],
    )
    @action(detail=True, methods=['get'], url_path='historial')
    def historial(self, request, pk=None):
        enc = self.get_object()
        qs = enc.historial.select_related('empleado').order_by('-fecha_cambio')
        paginator = HistorialPagination()
        page = paginator.paginate_queryset(qs, request)
        if page is not None:
            return paginator.get_paginated_response(
                HistorialEstadoSerializer(page, many=True).data
            )
        return Response(HistorialEstadoSerializer(qs, many=True).data)

    @extend_schema(
        summary='Estadísticas globales',
        description='Contadores del sistema: activas, en tránsito, con retraso y entregadas hoy.',

```

```

tags=['Encomiendas'],
responses={200: OpenApiResponse(description='Objeto con contadores')},
)
@action(detail=False, methods=['get'])
def estadisticas(self, request):
    from django.utils import timezone
    hoy = timezone.now().date()
    return Response({
        'total_activas': Encomienda.objects.activas().count(),
        'en_transito': Encomienda.objects.en_transito().count(),
        'con_retraso': Encomienda.objects.con_retraso().count(),
        'entregadas_hoy': Encomienda.objects.filter(
            estado='EN', fecha_entrega_real=hoy
        ).count(),
    })

```

## Paso 9 — Anotar las vistas genéricas

Abre `envios/api_views.py` y agrega las anotaciones a las vistas de clientes y rutas:

```

Python
# envios/api_views.py
from drf_spectacular.utils import extend_schema

@extend_schema(
    summary='Listar clientes activos',
    description='Devuelve todos los clientes con estado Activo, paginados de 20 en 20.',
    tags=['Clientes'],
)
class ClienteListView(generics.ListAPIView):
    serializer_class = ClienteSerializer
    permission_classes = [IsAuthenticated]
    pagination_class = ClientePagination

    def get_queryset(self):
        return Cliente.objects.activos()

```

```

@extend_schema(
    summary='Listar rutas activas',
    description='Devuelve todas las rutas con estado Activo. Sin paginacion.',
    tags=['Rutas'],
)
class RutaListView(generics.ListAPIView):
    serializer_class = RutaSerializer
    permission_classes = [IsAuthenticated]
    pagination_class = None

    def get_queryset(self):
        return Ruta.objects.activas()

```

## Paso 10 — Verificar el resultado en Swagger

Reinicia el contenedor y verifica que los cambios se reflejan en la documentación:

```

Python
# Reiniciar el contenedor para aplicar los cambios
docker compose restart web

# Verificar que el schema se genera sin errores
docker compose exec web python manage.py spectacular --validate
# Resultado esperado: 'Schema validated successfully!'

# Abrir Swagger UI
# http://localhost:8000/api/docs/

# Verificar que aparecen los siguientes grupos (tags):
# - Encomiendas:
#     GET    /api/v1/encomiendas/      'Listar encomiendas'
#     POST   /api/v1/encomiendas/      'Crear encomienda'
#     GET    /api/v1/encomiendas/{id}/ 'Detalle de encomienda'
#     PUT    /api/v1/encomiendas/{id}/ 'Actualizar encomienda'
#     PATCH  /api/v1/encomiendas/{id}/ 'Actualizar parcial'
#     DELETE /api/v1/encomiendas/{id}/ 'Eliminar encomienda'
#     POST   /api/v1/encomiendas/{id}/cambiar_estado/ 'Cambiar estado de
encomienda'

```

```
# GET /api/v1/encomiendas/{id}/historial/ 'Historial de estados'
# GET /api/v1/encomiendas/con_retraso/ 'Encomiendas con retraso'
# GET /api/v1/encomiendas/pendientes/ 'Encomiendas pendientes'
# GET /api/v1/encomiendas/estadisticas/ 'Estadisticas globales'
#
# - Clientes:
# GET /api/v1/clientes/ 'Listar clientes activos'
#
# - Rutas:
# GET /api/v1/rutas/ 'Listar rutas activas'
#
# - Auth:
# POST /api/v1/auth/token/ 'Obtener JWT'
# POST /api/v1/auth/token/refresh/ 'Renovar JWT'
```

## Paso 11 — Descargar el schema para uso externo

El schema OpenAPI 3.0 se puede descargar y usar para generar clientes HTTP en cualquier lenguaje o importar en Postman:

Python

```
# Descargar el schema en YAML (formato por defecto)
curl http://localhost:8000/api/schema/ \
  -H 'Authorization: Bearer $TOKEN' \
  -o encomiendas_api_schema.yaml

# Descargar el schema en JSON
curl 'http://localhost:8000/api/schema/?format=json' \
  -H 'Authorization: Bearer $TOKEN' \
  -o encomiendas_api_schema.json

# — Importar en Postman —————
# 1. Abrir Postman
# 2. Hacer clic en 'Import'
# 3. Seleccionar el archivo encomiendas_api_schema.yaml
# 4. Postman genera automaticamente todas las requests con sus bodies
```

```
# — Generar el schema desde el manage.py (sin servidor) —
docker compose exec web python manage.py spectacular --file schema.yml
# Genera el archivo schema.yml en la raiz del proyecto dentro del contenedor

# Copiar a la maquina local

docker compose cp web:/app/schema.yml ./schema.yml
```

## Swagger vs ReDoc — cuándo usar cada uno

| Característica     | Swagger UI                            | ReDoc                              |
|--------------------|---------------------------------------|------------------------------------|
| URL en el proyecto | http://localhost:8000/api/docs/       | http://localhost:8000/api/redoc/   |
| Interactividad     | Permite probar endpoints (Try it out) | Solo lectura                       |
| Autenticación      | Botón Authorize integrado             | No aplica                          |
| Presentación       | Más técnica, un endpoint por fila     | Más limpia, con descripción amplia |
| Ejemplos           | Muestra ejemplos inline               | Muestra ejemplos en panel lateral  |
| Mejor para         | Desarrollo y pruebas del equipo       | Documentar a clientes externos     |

## RESUMEN — ARCHIVOS MODIFICADOS EN ESTA SECCIÓN

config/settings.py      <- SPECTACULAR\_SETTINGS ampliado con TAGS y opciones

config/urls.py          <- sin cambios (ya tenia las rutas desde la instalacion)

envios/viewsets.py      <- se agrego @extend\_schema\_view en EncomiendaViewSet  
y @extend\_schema en cada @action:  
cambiar\_estado, con\_retrazo, pendientes,  
historial, estadisticas

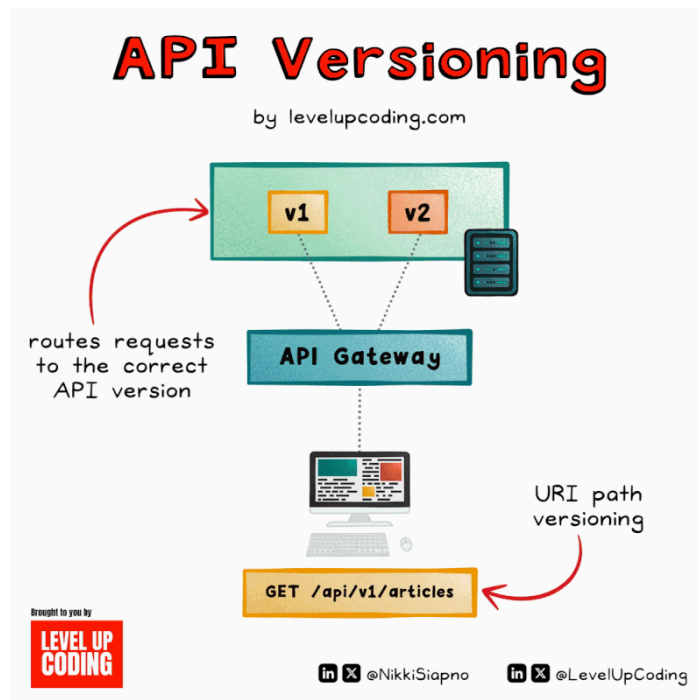
envios/api\_views.py      <- se agrego @extend\_schema en ClienteListView  
y RutaListView con tags y descripciones

Resultado en Swagger UI:

- 11 endpoints de encomiendas documentados con descripcion y ejemplos
- Tags organizando los endpoints por recurso
- Autenticacion JWT integrada en el boton Authorize
- Schema descargable en YAML/JSON para importar en Postman

## 6.8. API Versioning

El versionado de la API permite introducir cambios y mejoras sin romper los clientes que ya consumen la v1. En el sistema de encomiendas, la v1 devuelve solo los IDs de remitente y destinatario; la v2 devuelve los objetos anidados completos y campos adicionales de análisis. Ambas versiones conviven en paralelo.



QUÉ CAMBIA ENTRE V1 Y V2 EN NUESTRO PROYECTO

v1 (actual): respuesta del listado con IDs simples

```
{"id": 1, "codigo": "ENC-2026-001", "remitente": 5, "ruta": 2, ...}
```

v2 (nueva): respuesta con objetos anidados + campos de analisis

```
{  
    "id": 1, "codigo": "ENC-2026-001",  
    "remitente": {"id": 5, "nombre_completo": "Ramirez, Carlos", ...},  
    "ruta": {"codigo": "LIM-TRU", "destino": "Trujillo", ...},  
    "dias_en_transito": 3,  
    "descripcion_corta": "Documentos leg...",  
    "meta": {"version": "v2", "generado": "2026-05-07"}  
}
```

## Paso 1 — Configurar el versionado en settings.py

Abre `config/settings.py` y agrega las claves de versionado al bloque `REST_FRAMEWORK` que ya existe:

Python

```
# config/settings.py  
# Busca el bloque REST_FRAMEWORK y agrega las claves de versionado:  
  
REST_FRAMEWORK = {  
    # ... claves ya existentes ...  
    'DEFAULT_AUTHENTICATION_CLASSES': [...],  
    'DEFAULT_PERMISSION_CLASSES': [...],  
    'DEFAULT_PAGINATION_CLASS': '...',  
    'PAGE_SIZE': 15,  
    'DEFAULT_SCHEMA_CLASS': '...',  
  
    # — NUEVAS claves de versionado —————  
    'DEFAULT_VERSIONING_CLASS': (  
        'rest_framework.versioning.URLPathVersioning'  
    ),  
    'ALLOWED_VERSIONS': ['v1', 'v2'],    # versiones permitidas
```

```

'DEFAULT_VERSION': 'v1',          # version si no se especifica
'VERSION_PARAM': 'version',      # nombre del parametro en la URL
}

```

## Paso 2 — Actualizar config/urls.py para incluir el parámetro de versión

Abre `config/urls.py` y modifica la ruta de la API para incluir `<version>` como parámetro dinámico en la URL:

Python

```

# config/urls.py
# ANTES (lo que ya teníamos):
# path('api/v1/', include('api.urls')),

# DESPUES (con el parametro de version):
urlpatterns = [
    path('admin/', admin.site.urls),

    # Vistas web del sistema (sin cambios)
    path('', include('envios.urls')),
    path('accounts/', include('django.contrib.auth.urls')),

    # API REST con versionado dinamico
    # <version> captura 'v1' o 'v2' de la URL
    path('api/<version>/', include('api.urls')),

    # Auth JWT (no tiene version, es igual para todos)
    path('api/v1/auth/token/', TokenObtainPairView.as_view()),
    path('api/v1/auth/token/refresh/', TokenRefreshView.as_view()),

    # Documentacion
    path('api/schema/', SpectacularAPIView.as_view(), name='schema'),
    path('api/docs/', SpectacularSwaggerView.as_view(), name='swagger'),
    path('api/redoc/', SpectacularRedocView.as_view(), name='redoc'),
]

```

```

if settings.DEBUG:
    urlpatterns += static(settings.STATIC_URL,
document_root=settings.STATIC_ROOT)
    urlpatterns += static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT)

# Con esta configuracion ambas rutas funcionan:
# GET /api/v1/encomiendas/  <- version 1

# GET /api/v2/encomiendas/  <- version 2

```

## Paso 3 — Crear el serializer de v2

Abre `envios/serializers.py` y agrega el nuevo serializer para la v2. Lo ubicamos al final del archivo, después de todos los serializers existentes:

Python

```

# envios/serializers.py
# Agregar al FINAL del archivo, despues de HistorialEstadoSerializer

class EncomiendaV2Serializer(serializers.ModelSerializer):
    """
    Serializer para la API v2.
    Diferencias con v1:
    - remitente y destinatario como objetos anidados completos
    - ruta como objeto anidado
    - Campos de analisis: dias_en_transito, descripcion_corta
    - Campo 'meta' con informacion de la version
    """
    # Objetos anidados completos (en v1 son solo IDs)
    remitente = ClienteSerializer(read_only=True)
    destinatario = ClienteSerializer(read_only=True)
    ruta = RutaSerializer(read_only=True)

    # Para escritura: seguir aceptando IDs
    remitente_id = serializers.PrimaryKeyRelatedField(
        queryset=Cliente.objects.ativos(),

```

```

write_only=True, source='remitente'
)
destinatario_id = serializers.PrimaryKeyRelatedField(
    queryset=Cliente.objects.activos(),
    write_only=True, source='destinatario'
)
ruta_id = serializers.PrimaryKeyRelatedField(
    queryset=Ruta.objects.activas(),
    write_only=True, source='ruta'
)

# Campos nuevos en v2 (propiedades del modelo)
dias_en_transito = serializers.ReadOnlyField()
tiene_retraso = serializers.ReadOnlyField()
esta_entregada = serializers.ReadOnlyField()
descripcion_corta = serializers.ReadOnlyField()

# Campo de metadatos de la version
meta = serializers.SerializerMethodField()

class Meta:
    model = Encomienda
    fields = [
        'id', 'codigo', 'descripcion', 'descripcion_corta',
        'peso_kg', 'volumen_cm3', 'costo_envio',
        'remitente', 'remitente_id',
        'destinatario', 'destinatario_id',
        'ruta', 'ruta_id',
        'estado', 'fecha_registro', 'fecha_entrega_est',
        'dias_en_transito', 'tiene_retraso', 'esta_entregada',
        'observaciones', 'meta',
    ]
    read_only_fields = ['codigo', 'fecha_registro']

    def get_meta(self, obj):
        """Metadatos utiles para el cliente que consume la API"""
        from django.utils import timezone
        return {
            'version': 'v2',
            'generado': timezone.now().strftime('%Y-%m-%dT%H:%M:%SZ'),
            'puede_editar': not obj.esta_entregada,
        }

```

## Paso 4 — Actualizar el ViewSet para manejar ambas versiones

Abre `envios/viewsets.py` y actualiza `get_serializer_class()` y `get_queryset()` para que respondan distinto según la versión de la URL:

Python

```
# envios/viewsets.py
# Agregar el import del nuevo serializer
from .serializers import (
    EncomiendaSerializer,
    EncomiendaListSerializer,
    EncomiendaDetailSerializer,
    EncomiendaV2Serializer,    # <- NUEVO
    HistorialEstadoSerializer,
)

class EncomiendaViewSet(viewsets.ModelViewSet):
    queryset = Encomienda.objects.con_relaciones()
    permission_classes = [IsAuthenticated]
    pagination_class = EncomiendaPagination

    def get_serializer_class(self):
        """
        Elegir el serializer segun la version Y la accion.
        v1 list      -> EncomiendaListSerializer (ligero)
        v1 retrieve -> EncomiendaDetailSerializer (con anidados)
        v1 write     -> EncomiendaSerializer (estandar)
        v2 cualquier accion -> EncomiendaV2Serializer
        """
        version = getattr(self.request, 'version', 'v1')

        # v2: un solo serializer para todo
        if version == 'v2':
            return EncomiendaV2Serializer

        # v1: serializer segun la accion
```

```

if self.action == 'list':
    return EncomiendaListSerializer
if self.action == 'retrieve':
    return EncomiendaDetailSerializer
return EncomiendaSerializer

def get_queryset(self):
    """
    v1 y v2 usan el mismo queryset optimizado.
    Si en el futuro v2 necesita datos extra, se agrega aqui.
    """
    qs = Encomienda.objects.con_relaciones()

    estado = self.request.query_params.get('estado')
    if estado:
        qs = qs.filter(estado=estado)
    return qs

def list(self, request, *args, **kwargs):
    """Agregar cabecera X-API-Version en la respuesta"""
    response = super().list(request, *args, **kwargs)
    response['X-API-Version'] = getattr(request, 'version', 'v1')
    return response

def retrieve(self, request, *args, **kwargs):
    """Agregar cabecera X-API-Version en el detalle"""
    response = super().retrieve(request, *args, **kwargs)
    response['X-API-Version'] = getattr(request, 'version', 'v1')
    return response

# Las acciones @action no cambian: son iguales en v1 y v2
# cambiar_estado, con_retraso, pendientes, historial, estadisticas

# ... igual que antes ...

```

## Paso 5 — Verificar que ambas versiones responden



```
#      "remitente": {
#      "id": 5,
#      "nro_doc": "12345678",
#      "nombre_completo": "Ramirez Torres, Carlos",
#      "esta_activo": true
#      },
#      "ruta": {
#      "codigo": "LIM-TRU",
#      "origen": "Lima",
#      "destino": "Trujillo",
#      "precio_base": "25.00"
#      },
#      "dias_en_transito": 3,
#      "tiene_retraso": false,
#      "descripcion_corta": "Documentos leg...",
#      "meta": {
#      "version": "v2",
#      "generado": "2026-05-07T14:30:00Z",
#      "puede_editar": true
#      }
#      }
#  ]
# }
```

## Paso 6 — Verificar la cabecera X-API-Version

Python

### Verificar la cabecera X-API-Version

# La cabecera X-API-Version aparece en la respuesta de cada version

# v1: ver las cabeceras de respuesta

```
curl -I http://localhost:8000/api/v1/encomiendas/ \
-H "Authorization: Bearer $TOKEN"
```

# Cabeceras de respuesta:

# HTTP/1.1 200 OK

# Content-Type: application/json

```

# X-API-Version: v1      <- confirma la version en uso

# v2: ver las cabeceras de respuesta
curl -I http://localhost:8000/api/v2/encomiendas/ \
  -H "Authorization: Bearer $TOKEN"
# Cabeceras de respuesta:
# HTTP/1.1 200 OK
# Content-Type: application/json
# X-API-Version: v2      <- confirma la version en uso

# Version no permitida (devuelve 404):
curl -I http://localhost:8000/api/v3/encomiendas/ \
  -H "Authorization: Bearer $TOKEN"
# HTTP/1.1 404 Not Found

# (v3 no esta en ALLOWED_VERSIONS)

```

## Paso 7 — Leer la versión desde el shell

Puedes verificar que Django captura correctamente la versión desde el shell:

Python

### Django Shell – verificar comportamiento de versiones

```
docker compose exec web python manage.py shell
```

```

from django.test import RequestFactory
from rest_framework.versioning import URLPathVersioning

factory = RequestFactory()

# Simular un request a v1
request_v1 = factory.get('/api/v1/encomiendas/')
scheme = URLPathVersioning()

# El viewset lee la version de esta forma internamente:
# version = getattr(request, 'version', 'v1')
# DRF la extrae de la URL y la guarda en request.version

```

```

# Verificar con un test rapido desde el shell
from rest_framework.test import APIClient
from django.contrib.auth.models import User
from rest_framework_simplejwt.tokens import RefreshToken

user = User.objects.first()
refresh = RefreshToken.for_user(user)
token = str(refresh.access_token)

client = APIClient()
client.credentials(HTTP_AUTHORIZATION=f'Bearer {token}')

# Request a v1
resp_v1 = client.get('/api/v1/encomiendas/')
print('Version:', resp_v1['X-API-Version']) # v1
primer = resp_v1.data['results'][0]
print('Tipo remitente v1:', type(primer.get('remitente_nombre'))) # str

# Request a v2
resp_v2 = client.get('/api/v2/encomiendas/')
print('Version:', resp_v2['X-API-Version']) # v2
primer = resp_v2.data['results'][0]
print('Tipo remitente v2:', type(primer.get('remitente'))) # dict

print('Tiene meta:', 'meta' in primer) # True

```

## Paso 8 — Otros esquemas de versionado (referencia)

Aunque `URLPathVersioning` es el recomendado para este proyecto, DRF soporta otros esquemas que pueden ser útiles en distintos contextos:

| Esquema                               | Cómo se usa                                       | Cuándo elegirlo                                     |
|---------------------------------------|---------------------------------------------------|-----------------------------------------------------|
| <code>URLPathVersioning</code>        | <code>GET /api/v2/encomiendas/</code>             | Proyectos nuevos. URL clara, visible, cacheable     |
| <code>AcceptHeaderVersioning</code>   | <code>Accept: application/json; version=v2</code> | APIs públicas. URL limpia, version en cabecera      |
| <code>QueryParameterVersioning</code> | <code>GET /api/encomiendas/?version=v2</code>     | Prototipado rapido. Facil de probar en el navegador |

|                     |                                    |                                                          |
|---------------------|------------------------------------|----------------------------------------------------------|
| HostNameVersioning  | v2.api.encomiendas.pe/encomiendas/ | Grandes plataformas con subdominios por version          |
| NamespaceVersioning | URL con namespace Django           | Proyectos con URLs ya definidas que no se pueden cambiar |

Python

# Ejemplo: cambiar a AcceptHeaderVersioning (sin tocar las URLs)

# config/settings.py

```
REST_FRAMEWORK = {
    'DEFAULT_VERSIONING_CLASS': (
        'rest_framework.versioning.AcceptHeaderVersioning'
    ),
    'ALLOWED_VERSIONS': ['v1', 'v2'],
    'DEFAULT_VERSION': 'v1',
}
```

# config/urls.py: la URL vuelve a ser fija  
path('api/', include('api.urls')),

# Uso desde el cliente:

# v1 (por defecto, sin cabecera):

# GET /api/encomiendas/

# v2 (con cabecera Accept):

# GET /api/encomiendas/

# Accept: application/json; version=v2

# Con curl:

```
curl http://localhost:8000/api/encomiendas/ \
  -H "Authorization: Bearer $TOKEN" \
  -H "Accept: application/json; version=v2"
```

config/settings.py      <- se agregaron 4 claves al bloque REST\_FRAMEWORK:  
                          DEFAULT\_VERSIONING\_CLASS, ALLOWED\_VERSIONS,  
                          DEFAULT\_VERSION, VERSION\_PARAM

config/urls.py      <- se cambio 'api/v1/' por 'api/<version>/'  
                          para capturar la version dinamicamente

envios/serializers.py <- se agrego EncomiendaV2Serializer al final:  
                          remitente/destinatario/ruta como objetos anidados,  
                          dias\_en\_transito, tiene\_retraso, descripcion\_corta,  
                          campo 'meta' con version y puede\_editar

envios/viewsets.py      <- se actualizo get\_serializer\_class() para devolver  
                          EncomiendaV2Serializer cuando request.version == 'v2';  
                          se agrego cabecera X-API-Version en list() y retrieve()

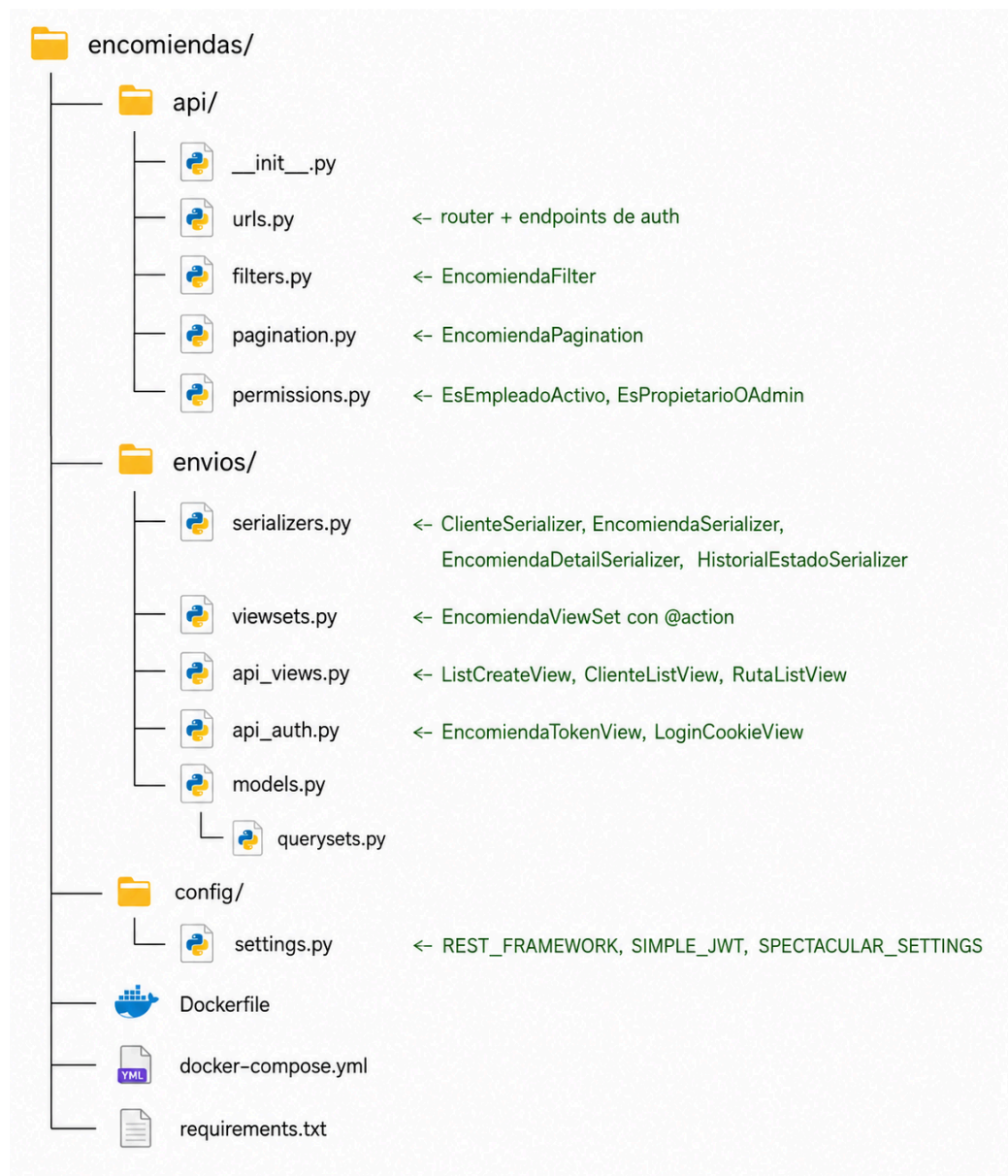
Resultado:

GET /api/v1/encomiendas/ -> EncomiendaListSerializer (IDs simples)

GET /api/v2/encomiendas/ -> EncomiendaV2Serializer (objetos anidados + meta)

Ambas versiones comparten el mismo ViewSet, Router y logica de negocio

## Estructura final del proyecto con DRF



## 6.9. Testing de APIs con DRF

Las pruebas automatizadas garantizan que la API funciona correctamente ante cambios en el código. En lugar de probar manualmente con curl o Swagger cada vez que se hace un cambio, los tests lo hacen en segundos. DRF provee APIClient para simular peticiones HTTP sin levantar un servidor real.

### QUÉ VAMOS A TESTEAR

Al terminar esta seccion tendremos 22 tests que verifican automaticamente:

- Autenticacion: sin token -> 401, token invalido -> 401, con token -> 200
- Listado: paginacion correcta, filtro por estado, busqueda por codigo
- Creacion: datos validos -> 201, remitente=destinatario -> 400, peso negativo -> 400, codigo sin ENC- -> 400, sin auth -> 401
- Cambio de estado: exitoso + verifica BD y historial, mismo estado -> 400
- Acciones: con\_retraso, pendientes y estadisticas
- Versionado: v1 y v2 responden distinto, v3 -> 404

## Paso 1 — Instalar las dependencias de testing

Python

```
# Agregar al requirements.txt
pytest-django==4.8.0
factory-boy==3.3.0
faker==24.0.0

# Reconstruir la imagen Docker
docker compose down
docker compose build
docker compose up -d

# Verificar la instalacion
```

```
docker compose exec web pip show pytest-django factory-boy faker
```

## Paso 2 — Crear pytest.ini en la raíz del proyecto

Crea el archivo `pytest.ini` en la raíz del proyecto (junto a `manage.py` y `docker-compose.yml`):

```
Python
# pytest.ini
[pytest]
DJANGO_SETTINGS_MODULE = config.settings
python_files = tests.py test_*.py *_tests.py
python_classes = *Tests
python_functions = test_*

addopts = -v --tb=short
```

### 6.9.3 Paso 3 — Crear la estructura de la carpeta de tests

None

```
Terminal – crear estructura de tests
# Crear la carpeta tests dentro de envios/
docker compose exec web mkdir envios/tests
docker compose exec web touch envios/tests/__init__.py
docker compose exec web touch envios/tests/factories.py
docker compose exec web touch envios/tests/test_api.py

# Verificar la estructura creada
docker compose exec web find envios/tests/ -type f
# envios/tests/__init__.py
# envios/tests/factories.py
# envios/tests/test_api.py
```

## 6.9.4 Paso 4 — Crear conftest.py con fixtures compartidos

Creas `conftest.py` en la raíz del proyecto (junto a `manage.py`). Los fixtures definidos aquí son accesibles por todos los tests:

Python

```
# conftest.py (raíz del proyecto)
import pytest
from rest_framework.test import APIClient
from rest_framework_simplejwt.tokens import RefreshToken
from django.contrib.auth.models import User

@pytest.fixture
def api_client():
    """Cliente de API sin autenticación"""
    return APIClient()

@pytest.fixture
def user(db):
    """Usuario de prueba"""
    return User.objects.create_user(
        username='test_empleado',
        email='empleado@encomiendas.pe',
        password='test1234',
    )

@pytest.fixture
def auth_client(api_client, user):
    """Cliente de API con JWT válido"""
    refresh = RefreshToken.for_user(user)
    api_client.credentials(
        HTTP_AUTHORIZATION=f'Bearer {refresh.access_token}'
    )

    return api_client
```

## 6.9.5 Paso 5 — Implementar las factories

Abre `envios/tests/factories.py` y escribe el contenido completo:

Python

```
# envios/tests/factories.py
import factory
from decimal import Decimal
from factory.django import DjangoModelFactory
from django.contrib.auth.models import User
from clientes.models import Cliente
from rutas.models import Ruta
from envios.models import Empleado, Encomienda
from config.choices import EstadoGeneral, TipoDocumento

class UserFactory(DjangoModelFactory):
    class Meta:
        model = User
        username = factory.Sequence(lambda n: f'user{n}')
        email = factory.LazyAttribute(lambda o: f'{o.username}@enc.pe')
        password = factory.PostGenerationMethodCall('set_password', 'test1234')

class ClienteFactory(DjangoModelFactory):
    class Meta:
        model = Cliente
        tipo_doc = TipoDocumento.DNI
        nro_doc = factory.Sequence(lambda n: f'{10000000 + n}')
        nombres = factory.Faker('first_name', locale='es_PE')
        apellidos = factory.Faker('last_name', locale='es_PE')
        estado = EstadoGeneral.ACTIVO

class RutaFactory(DjangoModelFactory):
    class Meta:
        model = Ruta
        codigo = factory.Sequence(lambda n: f'RUT-{n:03d}')
        origen = 'Lima'
        destino = factory.Sequence(lambda n: f'Ciudad-{n}')
        precio_base = Decimal('25.00')
        dias_entrega = 2
        estado = EstadoGeneral.ACTIVO

class EmpleadoFactory(DjangoModelFactory):
```

```

class Meta:
    model = Empleado
    codigo = factory.Sequence(lambda n: f'EMP-{n:03d}')
    nombres = factory.Faker('first_name', locale='es_PE')
    apellidos = factory.Faker('last_name', locale='es_PE')
    cargo = 'Operador de Envios'
    email = factory.LazyAttribute(lambda o: f'{o.codigo}@enc.pe')
    fecha_ingreso = factory.Faker('date_this_decade')
    estado = EstadoGeneral.ACTIVO

class EncomiendaFactory(DjangoModelFactory):
    class Meta:
        model = Encomienda
        codigo = factory.Sequence(lambda n: f'ENC-2026-{n:04d}')
        descripcion = factory.Faker('sentence', locale='es_PE')
        peso_kg = Decimal('3.50')
        remitente = factory.SubFactory(ClienteFactory)
        destinatario = factory.SubFactory(ClienteFactory)
        ruta = factory.SubFactory(RutaFactory)
        empleado_registro = factory.SubFactory(EmpleadoFactory)
        costo_envio = Decimal('25.00')
        estado = 'PE'

```

## 6.9.6 Paso 6 — Implementar test\_api.py (parte 1: auth, listado, crear)

Abre `envios/tests/test_api.py` y escribe el contenido completo:

```

Python
# envios/tests/test_api.py
import pytest
from decimal import Decimal
from rest_framework.test import APIClient
from rest_framework import status
from rest_framework_simplejwt.tokens import RefreshToken
from django.urls import reverse

from envios.models import Encomienda

```

```

from config.choices import EstadoEnvio
from .factories import (
    UserFactory, ClienteFactory, RutaFactory,
    EmpleadoFactory, EncomiendaFactory,
)

@pytest.mark.django_db
class TestAutenticacion:
    def test_sin_token_devuelve_401(self, api_client):
        response = api_client.get(reverse('encomienda-list'))
        assert response.status_code == status.HTTP_401_UNAUTHORIZED

    def test_token_invalido_devuelve_401(self, api_client):
        api_client.credentials(HTTP_AUTHORIZATION='Bearer tokeninvalido')
        response = api_client.get(reverse('encomienda-list'))
        assert response.status_code == status.HTTP_401_UNAUTHORIZED

    def test_con_token_valido_devuelve_200(self, auth_client):
        response = auth_client.get(reverse('encomienda-list'))
        assert response.status_code == status.HTTP_200_OK

@pytest.mark.django_db
class TestListadoEncomiendas:
    def setup_method(self):
        self.user = UserFactory()
        self.empleado = EmpleadoFactory(email=self.user.email)
        self.ruta = RutaFactory()
        self.cliente1 = ClienteFactory()
        self.cliente2 = ClienteFactory()
        self.client = APIClient()
        refresh = RefreshToken.for_user(self.user)
        self.client.credentials(
            HTTP_AUTHORIZATION=f'Bearer {refresh.access_token}'
        )

    def test_lista_respuesta_paginada(self):
        EncomiendaFactory(
            remitente=self.cliente1, destinatario=self.cliente2,
            ruta=self.ruta, empleado_registro=self.empleado
        )
        response = self.client.get(reverse('encomienda-list'))
        assert response.status_code == status.HTTP_200_OK

```

```

for campo in ['count', 'next', 'previous', 'results']:
    assert campo in response.data
assert response.data['count'] == 1

def test_filtro_por_estado(self):
    enc_pe = EncomiendaFactory(estado='PE', ruta=self.ruta,
                               remitente=self.cliente1, destinatario=self.cliente2,
                               empleado_registro=self.empleado)
    enc_tr = EncomiendaFactory(estado='TR', ruta=self.ruta,
                               remitente=self.cliente1, destinatario=self.cliente2,
                               empleado_registro=self.empleado)
    response = self.client.get(reverse('encomienda-list') + '?estado=PE')
    codigos = [r['codigo'] for r in response.data['results']]
    assert enc_pe.codigo in codigos
    assert enc_tr.codigo not in codigos

def test_busqueda_por_codigo(self):
    enc = EncomiendaFactory(codigo='ENC-2026-BUSCAR', ruta=self.ruta,
                             remitente=self.cliente1, destinatario=self.cliente2,
                             empleado_registro=self.empleado)
    response = self.client.get(reverse('encomienda-list') + '?search=BUSCAR')
    assert response.data['count'] == 1
    assert response.data['results'][0]['codigo'] == 'ENC-2026-BUSCAR'

```

@pytest.mark.django\_db

class TestCrearEncomienda:

```

    def setup_method(self):
        self.user = UserFactory()
        self.empleado = EmpleadoFactory(email=self.user.email)
        self.cliente1 = ClienteFactory()
        self.cliente2 = ClienteFactory()
        self.ruta = RutaFactory(precio_base=Decimal('25.00'))
        self.client = APIClient()
        refresh = RefreshToken.for_user(self.user)
        self.client.credentials(
            HTTP_AUTHORIZATION=f'Bearer {refresh.access_token}'
        )
        self.data_valida = {
            'codigo': 'ENC-2026-TEST', 'descripcion': 'Paquete de prueba',
            'peso_kg': '3.50', 'remitente_id': self.cliente1.pk,
            'destinatario_id': self.cliente2.pk,
            'ruta_id': self.ruta.pk, 'costo_envio': '25.00',
        }

```

```

def test_crear_exitoso_devuelve_201(self):
    response = self.client.post(
        reverse('encomienda-list'), self.data_valida, format='json'
    )
    assert response.status_code == status.HTTP_201_CREATED
    assert response.data['codigo'] == 'ENC-2026-TEST'
    assert response.data['estado'] == 'PE'
    assert Encomienda.objects.filter(codigo='ENC-2026-TEST').exists()

def test_crear_asigna_empleado_del_token(self):
    self.client.post(
        reverse('encomienda-list'), self.data_valida, format='json'
    )
    enc = Encomienda.objects.get(codigo='ENC-2026-TEST')
    assert enc.empleado_registro.email == self.user.email

def test_remitente_igual_destinatario_devuelve_400(self):
    data = {**self.data_valida, 'destinatario_id': self.cliente1.pk}
    response = self.client.post(
        reverse('encomienda-list'), data, format='json'
    )
    assert response.status_code == status.HTTP_400_BAD_REQUEST
    assert 'destinatario' in response.data or 'non_field_errors' in
response.data

def test_peso_negativo_devuelve_400_con_campo(self):
    data = {**self.data_valida, 'peso_kg': '-1.00'}
    response = self.client.post(
        reverse('encomienda-list'), data, format='json'
    )
    assert response.status_code == status.HTTP_400_BAD_REQUEST
    assert 'peso_kg' in response.data

def test_codigo_sin_prefijo_devuelve_400(self):
    data = {**self.data_valida, 'codigo': 'PKG-2026-001'}
    response = self.client.post(
        reverse('encomienda-list'), data, format='json'
    )
    assert response.status_code == status.HTTP_400_BAD_REQUEST
    assert 'codigo' in response.data

def test_sin_auth_no_crea_y_devuelve_401(self, api_client):
    response = api_client.post(

```

```

        reverse('encomienda-list'), self.data_valida, format='json'
    )
    assert response.status_code == status.HTTP_401_UNAUTHORIZED

    assert not Encomienda.objects.filter(codigo='ENC-2026-TEST').exists()

```

## 6.9.6b Tests: cambiar estado, acciones y versionado

Python

```

# envios/tests/test_api.py (continua en el mismo archivo)

@pytest.mark.django_db
class TestCambiarEstado:
    """Tests del endpoint POST /api/v1/encomiendas/{pk}/cambiar_estado/"""

    def setup_method(self):
        self.user = UserFactory()
        self.Empleado = EmpleadoFactory(email=self.user.email)
        self.enc = EncomiendaFactory(
            empleado_registro=self.Empleado, estado='PE'
        )
        self.client = APIClient()
        refresh = RefreshToken.for_user(self.user)
        self.client.credentials(
            HTTP_AUTHORIZATION=f'Bearer {refresh.access_token}'
        )

    def test_cambiar_estado_exitoso_actualiza_bd_y_crea_historial(self):
        url = reverse('encomienda-cambiar-estado',
                      kwargs={'pk': self.enc.pk})
        data = {'estado': 'TR', 'observacion': 'Recogido en agencia Lima'}
        response = self.client.post(url, data, format='json')
        assert response.status_code == status.HTTP_200_OK
        assert response.data['estado'] == 'TR'
        self.enc.refresh_from_db()

```

```

assert self.enc.estado == EstadoEnvio.EN_TRANSITO
assert self.enc.historial.count() == 1
h = self.enc.historial.first()
assert h.estado_anterior == 'PE'
assert h.estado_nuevo == 'TR'
assert h.observacion == 'Recogido en agencia Lima'

def test_cambiar_al_mismo_estado_devuelve_400(self):
    url = reverse('encomienda-cambiar-estado', kwargs={'pk': self.enc.pk})
    response = self.client.post(url, {'estado': 'PE'}, format='json')
    assert response.status_code == status.HTTP_400_BAD_REQUEST
    self.enc.refresh_from_db()
    assert self.enc.historial.count() == 0 # no se creo historial

def test_sin_campo_estado_devuelve_400(self):
    url = reverse('encomienda-cambiar-estado', kwargs={'pk': self.enc.pk})
    response = self.client.post(url, {}, format='json')
    assert response.status_code == status.HTTP_400_BAD_REQUEST
    assert 'error' in response.data

def test_encomienda_inexistente_devuelve_404(self):
    url = reverse('encomienda-cambiar-estado', kwargs={'pk': 99999})
    response = self.client.post(url, {'estado': 'TR'}, format='json')
    assert response.status_code == status.HTTP_404_NOT_FOUND

```

@pytest.mark.django\_db

class TestAccionesPersonalizadas:

"""Tests de con\_retraso, pendientes y estadísticas"""

```

def setup_method(self):
    from django.utils import timezone
    from datetime import timedelta
    self.user = UserFactory()
    self.empleado = EmpleadoFactory(email=self.user.email)
    self.client = APIClient()
    refresh = RefreshToken.for_user(self.user)
    self.client.credentials(
        HTTP_AUTHORIZATION=f'Bearer {refresh.access_token}'
    )
    ayer = timezone.now().date() - timedelta(days=1)
    self.enc_retraso = EncomiendaFactory(
        estado='TR', fecha_entrega_est=ayer,
        empleado_registro=self.empleado
    )

```

```

)
self.enc_normal = EncomiendaFactory(
    estado='PE', empleado_registro=self.empleado
)

def test_con_retraso_solo_devuelve_retrasadas(self):
    response = self.client.get(reverse('encomienda-con-retraso'))
    assert response.status_code == status.HTTP_200_OK
    codigos = [r['codigo'] for r in response.data]
    assert self.enc_retraso.codigo in codigos
    assert self.enc_normal.codigo not in codigos

def test_pendientes_solo_devuelve_pendientes(self):
    response = self.client.get(reverse('encomienda-pendientes'))
    assert response.status_code == status.HTTP_200_OK
    codigos = [r['codigo'] for r in response.data]
    assert self.enc_normal.codigo in codigos
    assert self.enc_retraso.codigo not in codigos

def test_estadisticas_devuelve_todos_los_contadores(self):
    response = self.client.get(reverse('encomienda-estadisticas'))
    assert response.status_code == status.HTTP_200_OK
    for campo in ['total_activas', 'en_transito', 'con_retraso', 'entregadas_hoy']:
        assert campo in response.data
    assert response.data['con_retraso'] == 1

```

@pytest.mark.django\_db

class TestVersionado:

```

    """Tests de versionado v1 vs v2"""

    def setup_method(self):
        self.user = UserFactory()
        EmpleadoFactory(email=self.user.email)
        EncomiendaFactory()
        self.client = APIClient()
        refresh = RefreshToken.for_user(self.user)
        self.client.credentials(
            HTTP_AUTHORIZATION=f'Bearer {refresh.access_token}'
        )

    def test_v1_responde_200_con_cabecera(self):
        response = self.client.get('/api/v1/encomiendas/')
        assert response.status_code == status.HTTP_200_OK

```

```

assert response['X-API-Version'] == 'v1'

def test_v2_responde_200_con_cabecera(self):
    response = self.client.get('/api/v2/encomiendas/')
    assert response.status_code == status.HTTP_200_OK
    assert response['X-API-Version'] == 'v2'

def test_v2_incluye_campo_meta(self):
    response = self.client.get('/api/v2/encomiendas/')
    primer = response.data['results'][0]
    assert 'meta' in primer
    assert primer['meta']['version'] == 'v2'

def test_v3_no_permitida_devuelve_404(self):
    response = self.client.get('/api/v3/encomiendas/')

    assert response.status_code == status.HTTP_404_NOT_FOUND

```

## 6.9.7 Paso 7 — Ejecutar los tests

Python

**Terminal – resultado de pytest**

# Ejecutar TODOS los tests

docker compose exec web pytest

# Resultado esperado:

# ===== test session starts =====

# collected 22 items

#

# envios/tests/test\_api.py::TestAutenticacion::test\_sin\_token\_devuelve\_401 PASSED

# envios/tests/test\_api.py::TestAutenticacion::test\_token\_invalida\_devuelve\_401  
PASSED

# envios/tests/test\_api.py::TestAutenticacion::test\_con\_token\_valido\_devuelve\_200  
PASSED

# envios/tests/test\_api.py::TestListadoEncomiendas::test\_lista\_respuesta\_paginada  
PASSED

#

envios/tests/test\_api.py::TestListadoEncomiendas::test\_filtro\_por\_estado\_pendiente  
PASSED

```
# envios/tests/test_api.py::TestListadoEncomiendas::test_busqueda_por_codigo
PASSED
# envios/tests/test_api.py::TestCrearEncomienda::test_crear_exitoso_devuelve_201
PASSED
#
envios/tests/test_api.py::TestCrearEncomienda::test_crear_asigna_empleado_del_token PASSED
#
envios/tests/test_api.py::TestCrearEncomienda::test_crear_remitente_igual_destinatario_devuelve_400 PASSED
#
envios/tests/test_api.py::TestCrearEncomienda::test_crear_peso_negativo_devuelve_400 PASSED
#
envios/tests/test_api.py::TestCrearEncomienda::test_crear_codigo_sin_prefijo_devuelve_400 PASSED
# envios/tests/test_api.py::TestCrearEncomienda::test_crear_sin_auth_devuelve_401
PASSED
# ... (22 tests en total)
#
# ===== 22 passed in 4.2s =====
```

## 6.9.8 Paso 8 — Comandos útiles para el día a día

Python

### Comandos utiles de pytest

# Solo una clase de tests

```
docker compose exec web pytest envios/tests/test_api.py::TestCrearEncomienda -v
```

# Solo un test especifico

```
docker compose exec web pytest
envios/tests/test_api.py::TestCambiarEstado::test_cambiar_estado_exitoso_actualiza_bd_y_crea_historial -v
```

# Tests que contengan una palabra en el nombre

```
docker compose exec web pytest -k 'estado' -v
```

```

# Detener al primer fallo
docker compose exec web pytest -x

# Ver print() de los tests
docker compose exec web pytest -s

# Reporte de cobertura (requiere pytest-cov en requirements.txt)
docker compose exec web pytest --cov=envios --cov-report=term-missing

# Re-ejecutar solo los que fallaron la ultima vez
docker compose exec web pytest --lf

```

## 6.9.9 Paso 9 — Verificar que los tests detectan regresiones

Python

**Demostrar que los tests detectan cambios incorrectos**

```

# Prueba: comentar la validacion de remitente != destinatario
# en envios/serializers.py dentro del metodo validate()

```

```

# ANTES (correcto):
# if data.get('remitente') == data.get('destinatario'):
#     raise serializers.ValidationError({'destinatario': '...'})

```

```

# DESPUES (incorrecto, comentar para simular el error):
# # if data.get('remitente') == data.get('destinatario'):
# #     raise serializers.ValidationError(...)

```

# Ejecutar el test afectado:

```

docker compose exec web pytest
envios/tests/test_api.py::TestCrearEncomienda::test_crear_remitente_igual_destinat
ario_devuelve_400 -v

```

```

# Resultado: el test FALLA
# FAILED test_crear_remitente_igual_destinatario_devuelve_400
# AssertionError: assert 201 == 400
# El test detecto que la validacion fue removida

```

```
# Restaurar el código y volver a correr:
docker compose exec web pytest envios/tests/test_api.py::TestCrearEncomienda -v

# 6 passed
```

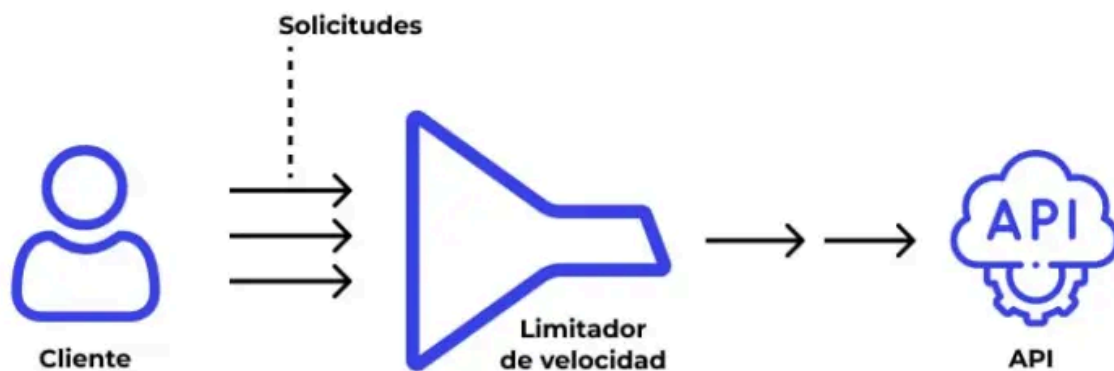
#### RESUMEN — ARCHIVOS CREADOS EN ESTA SECCIÓN

|                           |                                                                                    |
|---------------------------|------------------------------------------------------------------------------------|
| requirements.txt          | <- pytest-django, factory-boy, faker                                               |
| pytest.ini                | <- configuracion de pytest (raiz del proyecto)                                     |
| conftest.py               | <- fixtures: api_client, user, auth_client<br>(raiz del proyecto)                  |
| envios/tests/__init__.py  |                                                                                    |
| envios/tests/factories.py | <- UserFactory, ClienteFactory, RutaFactory,<br>EmpleadoFactory, EncomiendaFactory |
| envios/tests/test_api.py  | <- 22 tests en 6 clases:                                                           |
|                           | TestAutenticacion (3 tests)                                                        |
|                           | TestListadoEncomiendas (3 tests)                                                   |
|                           | TestCrearEncomienda (6 tests)                                                      |
|                           | TestCambiarEstado (4 tests)                                                        |
|                           | TestAccionesPersonalizadas (3 tests)                                               |
|                           | TestVersionado (4 tests)                                                           |

—

## 6.10. Throttling — Control de tasa de peticiones

El throttling limita el número de peticiones que un cliente puede hacer en un período de tiempo. Protege la API contra abuso, scraping y ataques de fuerza bruta. DRF lo incluye sin instalar nada extra.



Para más detalles puedes revisar el siguiente recurso:

<https://lab.wallarm.com/what/limitacion-de-velocidad/?lang=es>

### 6.10.1 Configuración global

Python

```
# config/settings.py
REST_FRAMEWORK = {
    'DEFAULT_THROTTLE_CLASSES': [
        'rest_framework.throttling.AnonRateThrottle',
        'rest_framework.throttling.UserRateThrottle',
    ],
    'DEFAULT_THROTTLE_RATES': {
```

```

        'anon': '20/hour',          # 20 requests/hora para no autenticados
        'user': '500/hour',        # 500 requests/hora para autenticados
        'login_attempt': '5/min',  # solo para el endpoint de login
    }
}

```

## 6.10.2 Throttles personalizados para encomiendas

Python

```

# api/throttles.py
from rest_framework.throttling import UserRateThrottle, AnonRateThrottle

class LoginRateThrottle(AnonRateThrottle):
    """Limitar intentos de login: 5 por minuto"""
    scope = 'login_attempt'

class EmpleadoRateThrottle(UserRateThrottle):
    """Empleados: 100 peticiones por minuto"""
    scope = 'empleado'

class CambioEstadoThrottle(UserRateThrottle):
    """Limitar cambios de estado: 30 por hora"""
    scope = 'cambio_estado'

# config/settings.py
REST_FRAMEWORK = {
    'DEFAULT_THROTTLE_RATES': {
        'anon': '20/hour',
        'user': '500/hour',
        'empleado': '100/min',
        'cambio_estado': '30/hour',
        'login_attempt': '5/min',
    }
}

```

```

# Aplicar en el ViewSet
from api.throttles import EmpleadoRateThrottle, CambioEstadoThrottle

class EncomiendaViewSet(viewsets.ModelViewSet):
    throttle_classes = [EmpleadoRateThrottle]

    def get_throttles(self):
        """Throttle diferente para la acción cambiar_estado"""
        if self.action == 'cambiar_estado':
            return [CambioEstadoThrottle()]
        return super().get_throttles()

# Aplicar en el endpoint de login
from api.throttles import LoginRateThrottle

class EncomiendaTokenView(TokenObtainPairView):
    throttle_classes = [LoginRateThrottle]
    serializer_class = EncomiendaTokenSerializer

# Respuesta cuando se supera el límite (status 429):
# {
#   'detail': 'Request was throttled. Expected available in 42 seconds.'
# }

# Header: Retry-After: 42

```

## 6.11. Manejo de Errores Personalizado

Por defecto DRF devuelve errores con formatos distintos según el tipo de error. Un exception handler personalizado garantiza que TODA la API devuelve errores con el mismo formato JSON, facilitando el manejo en el cliente.

### 6.11.1 Exception handler personalizado

Python

```
# api/exceptions.py
from rest_framework.views import exception_handler
from rest_framework.response import Response
from rest_framework import status
import logging

logger = logging.getLogger(__name__)

def encomiendas_exception_handler(exc, context):
    """
    Handler global de errores para la API de encomiendas.
    Devuelve siempre el mismo formato:
    {
    'error': True,
    'code': 'VALIDATION_ERROR',
    'message': 'Descripción legible del error',
    'detail': { ...errores por campo... }
    }
    """
    # Primero procesar con el handler por defecto de DRF
    response = exception_handler(exc, context)

    if response is not None:
        # Determinar el código semántico del error
        error_code = 'API_ERROR'
        message = 'Ha ocurrido un error procesando la solicitud.'

    if response.status_code == status.HTTP_400_BAD_REQUEST:
```

```

        error_code = 'VALIDATION_ERROR'
        message     = 'Los datos enviados contienen errores de validación.'
    elif response.status_code == status.HTTP_401_UNAUTHORIZED:
        error_code = 'AUTHENTICATION_REQUIRED'
        message     = 'Se requiere autenticación para acceder a este
recurso.'
    elif response.status_code == status.HTTP_403_FORBIDDEN:
        error_code = 'PERMISSION_DENIED'
        message     = 'No tienes permiso para realizar esta acción.'
    elif response.status_code == status.HTTP_404_NOT_FOUND:
        error_code = 'NOT_FOUND'
        message     = 'El recurso solicitado no existe.'
    elif response.status_code == status.HTTP_429_TOO_MANY_REQUESTS:
        error_code = 'RATE_LIMIT_EXCEEDED'
        message     = 'Se excedió el límite de solicitudes. Intenta más
tarde.'

    response.data = {
        'error': True,
        'code': error_code,
        'message': message,
        'detail': response.data,
    }
    return response

# Error no controlado: loggear y devolver 500
logger.error(
    f'Error no controlado en {context["view"].__class__.__name__}: {exc}',
    exc_info=True
)
return Response({
    'error': True,
    'code': 'INTERNAL_ERROR',
    'message': 'Error interno del servidor.',
    'detail': None,
}, status=status.HTTP_500_INTERNAL_SERVER_ERROR)

```

Modificamos el archivo de settings.py

```

Python
# config/settings.py

```

```
REST_FRAMEWORK = {
    'EXCEPTION_HANDLER': 'api.exceptions.encomiendas_exception_handler',
}
```

## 6.11.2 Excepciones personalizadas de negocio

Python

```
# api/exceptions.py
from rest_framework.exceptions import APIException
from rest_framework import status

class EstadoInvalidoError(APIException):
    status_code = status.HTTP_422_UNPROCESSABLE_ENTITY
    default_code = 'ESTADO_INVALIDO'
    default_detail = 'La transición de estado no está permitida.'

class EncomiendaYaEntregadaError(APIException):
    status_code = status.HTTP_409_CONFLICT
    default_code = 'YA_ENTREGADA'
    default_detail = 'La encomienda ya fue entregada y no puede modificarse.'

# Uso en el ViewSet
from api.exceptions import EstadoInvalidoError, EncomiendaYaEntregadaError

class EncomiendaViewSet(viewsets.ModelViewSet):

    @action(detail=True, methods=['post'], url_path='cambiar_estado')
    def cambiar_estado(self, request, pk=None):
        enc = self.get_object()

        if enc.esta_entregada:
            raise EncomiendaYaEntregadaError()

        try:
            enc.cambiar_estado(...)
```

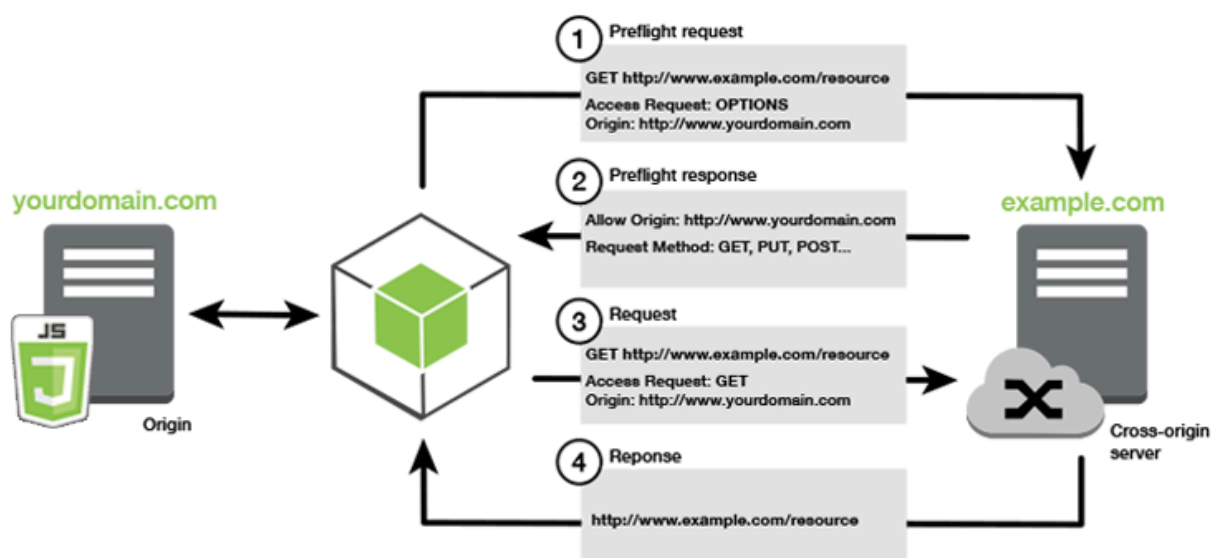


```
except ValueError as e:  
    raise EstadoInvalidoError(detail=str(e))
```

## 6.12. CORS — Cross-Origin Resource Sharing

Cuando el frontend (React, Vue, Angular) corre en un dominio o puerto distinto al de Django, el navegador bloquea las peticiones HTTP. CORS es el mecanismo que le dice al navegador qué orígenes externos pueden acceder a la API. Para mas detalle puedes revisar este recurso:

<https://lo-victoria.com/introduction-to-cross-origin-resource-sharing-cors>



Vamos a configurar CORS

Instalamos el paquete de cors en django

```
None
pip install django-cors-headers
```

Luego en config/[settings.py](#) modificamos algunos valroes

```
Python

# config/settings.py
INSTALLED_APPS = [
```

```

    ...
    'corsheaders',
]

MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware',    # <- PRIMERO en la lista
    'django.middleware.common.CommonMiddleware',
    ...
]

# — Configuración de desarrollo —————
# Permitir cualquier origen (SOLO EN DESARROLLO)
CORS_ALLOW_ALL_ORIGINS = True

# — Configuración de producción —————
CORS_ALLOW_ALL_ORIGINS = False
CORS_ALLOWED_ORIGINS = [
    'https://encomiendas-frontend.vercel.app',
    'https://admin.encomiendas.pe',
    'http://localhost:3000',    # React en desarrollo
    'http://localhost:5173',    # Vite en desarrollo
]

# Permitir envío de cookies (necesario para HttpOnly JWT)
CORS_ALLOW_CREDENTIALS = True

# Cabeceras y métodos permitidos
CORS_ALLOW_HEADERS = [
    'accept',
    'authorization',
    'content-type',
    'x-csrftoken',
    'x-requested-with',
]

CORS_ALLOW_METHODS = [
    'DELETE', 'GET', 'OPTIONS', 'PATCH', 'POST', 'PUT',
]

```

## 6.13. Serializers Avanzados

Los serializers tienen métodos que permiten personalizar completamente cómo se convierte un objeto a JSON y cómo se convierten los datos entrantes antes de validarlos. También es posible crear serializers que operen sobre múltiples objetos a la vez. En esta sección implementamos estas técnicas directamente en el proyecto de encomiendas.

### QUÉ IMPLEMENTAREMOS

```
to_representation() <- personalizar la salida JSON segun el rol del usuario
to_internal_value() <- normalizar datos entrantes antes de validar
EncomiendaBulkSerializer <- crear multiples encomiendas en una peticion
bulk_create action <- POST /api/v1/encomiendas/bulk_create/
bulk_estado action <- PATCH /api/v1/encomiendas/bulk_estado/
```

### 6.13.1 to\_representation() — personalizar la salida JSON

El método `to_representation()` se ejecuta cada vez que DRF serializa un objeto a JSON. Se usa para agregar campos calculados, ocultar campos según el rol del usuario, o cambiar el formato de los datos. Recibe la instancia del modelo y devuelve un diccionario.

### Paso 1 — Agregar to\_representation() al EncomiendaSerializer

Abre `envios/serializers.py` y agrega el método dentro de `EncomiendaSerializer`. Va después del `get_estado_display()` y antes del cierre de la clase:

Python

```
# envios/serializers.py
```

```

# Modificar EncomiendaSerializer: agregar to_representation()

class EncomiendaSerializer(serializers.ModelSerializer):
    esta_entregada = serializers.ReadOnlyField()
    tiene_retraso = serializers.ReadOnlyField()
    dias_en_transito = serializers.ReadOnlyField()
    descripcion_corta = serializers.ReadOnlyField()
    estado_display = serializers.SerializerMethodField()

    class Meta:
        model = Encomienda
        fields = [...]
        read_only_fields = ['codigo', 'fecha_registro']

    def get_estado_display(self, obj):
        return obj.get_estado_display()

# — NUEVO: personalizar la salida JSON —————
def to_representation(self, instance):
    """
    Se ejecuta al serializar (convertir objeto -> JSON).
    Permite modificar la respuesta despues de que DRF la genera.
    """
    data = super().to_representation(instance)

    # 1. Agregar campos de conveniencia calculados desde la ruta
    # (evita que el cliente tenga que hacer otra peticion)
    if instance.ruta_id:
        data['ruta_codigo'] = instance.ruta.codigo
        data['ruta_destino'] = instance.ruta.destino
        data['ruta_origen'] = instance.ruta.origen

    # 2. Formatear el costo con prefijo de moneda
    data['costo_display'] = f'S/ {instance.costo_envio:.2f}'

    # 3. Ocultar campos sensibles para usuarios no staff
    # Los empleados normales NO ven las observaciones internas
    # ni quien registro la encomienda
    request = self.context.get('request')
    if request and not request.user.is_staff:
        data.pop('observaciones', None)
        data.pop('empleado_registro', None)

    # 4. Agregar indicador visual del estado para el frontend

```

```

colores = {
    'PE': 'gray',
    'TR': 'blue',
    'DE': 'orange',
    'EN': 'green',
    'DV': 'red',
}
data['estado_color'] = colores.get(instance.estado, 'gray')

return data

```

## Paso 2 — Verificar to\_representation() con curl

Python

```

# Reiniciar el contenedor
docker compose restart web

# Pedir el detalle de una encomienda
curl http://localhost:8000/api/v1/encomiendas/1/ \
  -H "Authorization: Bearer $TOKEN" | python3 -m json.tool

# La respuesta ahora incluye los campos nuevos:
# {
#   "id": 1,
#   "codigo": "ENC-2026-001",
#   "estado": "TR",
#   "estado_display": "En transito",
#   "estado_color": "blue",      <- NUEVO
#   "ruta_codigo": "LIM-TRU",    <- NUEVO
#   "ruta_destino": "Trujillo",  <- NUEVO
#   "ruta_origen": "Lima",       <- NUEVO
#   "costo_display": "S/ 25.00", <- NUEVO
#   "observaciones": null,      <- oculto para no-staff
#   ...
# }

```

```
# Probar con un usuario que sea staff:
# El campo 'observaciones' y 'empleado_registro' aparecen

# Probar con un usuario no-staff:

# Los campos 'observaciones' y 'empleado_registro' NO aparecen
```

## Paso 3 — Agregar to\_internal\_value() para normalizar entradas

El método `to_internal_value()` se ejecuta antes de la validación, al recibir datos de una petición POST o PUT. Se usa para normalizar y limpiar los datos antes de que lleguen a los validators. Agrega este método en la misma clase:

Python

```
envios/serializers.py - to_internal_value()
# envios/serializers.py
# Agregar to_internal_value() dentro de EncomiendaSerializer

class EncomiendaSerializer(serializers.ModelSerializer):
    # ... campos y to_representation() ya definidos ...

    # — NUEVO: normalizar datos entrantes —————
    def to_internal_value(self, data):
        """
        Se ejecuta al deserializar (convertir JSON -> objeto Python).
        Se usa para limpiar y normalizar ANTES de la validacion.
        """

        # Necesitamos una copia mutable del dict
        if hasattr(data, '_mutable'):
            data._mutable = True
        data = data.copy() if hasattr(data, 'copy') else dict(data)

        # 1. Normalizar el codigo: forzar mayusculas y quitar espacios
        # Permite que el cliente envíe 'enc-2026-001' y lo corregimos
```

```

if 'codigo' in data and data['codigo']:
    data['codigo'] = str(data['codigo']).upper().strip()

# 2. Limpiar la descripcion: quitar espacios extra
if 'descripcion' in data and data['descripcion']:
    data['descripcion'] = str(data['descripcion']).strip()

# 3. Asegurar que el costo tenga maximo 2 decimales
# Permite que el cliente envíe '25' y lo normalizamos a '25.00'
if 'costo_envio' in data and data['costo_envio']:
    try:
        from decimal import Decimal, ROUND_HALF_UP
        costo = Decimal(str(data['costo_envio']))
        data['costo_envio'] = str(costo.quantize(Decimal('0.01'),
rounding=ROUND_HALF_UP))
    except Exception:
        pass # si falla, la validacion posterior lo rechazara

return super().to_internal_value(data)

```

## Paso 4 — Verificar to\_internal\_value() desde el shell

Python

**Verificar to\_internal\_value() en el shell**

`docker compose exec web python manage.py shell`

```

from envios.serializers import EncomiendaSerializer
from clientes.models import Cliente
from rutas.models import Ruta

```

```

c1 = Cliente.objects.first()
c2 = Cliente.objects.exclude(pk=c1.pk).first()
r1 = Ruta.objects.first()

```

```

# Enviar codigo en minusculas (el cliente lo envio mal)
data = {
    'codigo':          'enc-2026-test',      # minusculas
    'descripcion':     '  Ropa y calzado  ', # espacios extra
    'peso_kg':         '3.5',
    'remitente_id':    c1.pk,
    'destinatario_id': c2.pk,
    'ruta_id':         r1.pk,
    'costo_envio':     '25',                 # sin decimales
}

serializer = EncomiendaSerializer(data=data)
serializer.is_valid()    # dispara to_internal_value() + validadores

# Ver los datos despues de la normalizacion
print(serializer.validated_data['codigo'])
# ENC-2026-TEST    <- a mayusculas automaticamente

print(repr(serializer.validated_data.get('descripcion', '')))

# 'Ropa y calzado'    <- sin espacios al inicio y al final

```

## Paso 5 — Crear EncomiendaBulkSerializer

El `ListSerializer` es una clase especial de DRF que maneja listas de objetos. Al configurar `list_serializer_class` en el `Meta` del serializer, DRF lo activa automáticamente cuando se llama con `many=True`. Agrega esto en `envios/serializers.py` después de `EncomiendaSerializer`:

Python

```

envios/serializers.py - EncomiendaBulkSerializer
# envios/serializers.py
# Agregar ANTES de EncomiendaSerializer (porque este lo referencia)

class EncomiendaBulkSerializer(serializers.ListSerializer):
    """
    Serializer para operaciones masivas.

```

Se activa automaticamente cuando se usa EncomiendaSerializer(many=True).  
Reemplaza los metodos create() y update() por versiones optimizadas  
que hacen una sola query SQL en lugar de N queries.

"""

```
def create(self, validated_data):
```

"""

Crear multiples encomiendas con una sola query SQL.

Sin esto: Django hace INSERT por cada encomienda (N queries).

Con bulk\_create: Django hace 1 solo INSERT con todos los registros.

"""

```
encomiendas = [  
    Encomienda(**item) for item in validated_data  
]
```

```
return Encomienda.objects.bulk_create(encomiendas)
```

```
def update(self, instances, validated_data):
```

"""

Actualizar multiples encomiendas.

Nota: bulk\_update no dispara signals ni el metodo save() del modelo.

"""

```
instance_map = {enc.id: enc for enc in instances}  
updated = []
```

```
for item in validated_data:  
    enc_id = item.pop('id', None)  
    enc = instance_map.get(enc_id)  
    if enc:  
        for campo, valor in item.items():  
            setattr(enc, campo, valor)  
        updated.append(enc)
```

```
if updated:  
    Encomienda.objects.bulk_update(  
        updated,  
        ['estado', 'observaciones', 'costo_envio'],  
    )
```

```
return updated
```

# Modificar EncomiendaSerializer para activar el bulk serializer

```
class EncomiendaSerializer(serializers.ModelSerializer):
```

```
    # ... campos ya definidos ...
```

```

class Meta:
    model = Encomienda
    fields = [...]
    read_only_fields = ['codigo', 'fecha_registro']
    # Esta linea activa EncomiendaBulkSerializer cuando se usa many=True
    list_serializer_class = EncomiendaBulkSerializer

# ... resto de la clase ...

```

## Paso 6 — Agregar las acciones bulk al ViewSet

Abre `envios/viewsets.py` y agrega las dos acciones bulk al final de `EncomiendaViewSet`, junto a las otras acciones `@action` ya existentes:

Python

```

# envios/viewsets.py
# Agregar dentro de EncomiendaViewSet, despues de la accion estadisticas()

# — Bulk create: crear multiples encomiendas —————
@extend_schema(
    summary='Crear multiples encomiendas',
    description='Crea varias encomiendas en una sola peticion. Body: lista de
objetos.',
    tags=['Encomiendas'],
)
@action(detail=False, methods=['post'], url_path='bulk_create')
def bulk_create(self, request):
    """
    POST /api/v1/encomiendas/bulk_create/
    Body: [{enc1}, {enc2}, {enc3}]
    Crea todas las encomiendas con una sola query SQL.
    """
    # many=True activa EncomiendaBulkSerializer automaticamente

```

```

        serializer = self.get_serializer(
            data=request.data, many=True
        )
        if not serializer.is_valid():
            return Response(serializer.errors,
                status=status.HTTP_400_BAD_REQUEST)

        # Asignar el empleado a todas las encomiendas
        empleado = Empleado.objects.get(email=request.user.email)
        encomiendas = serializer.save(empleado_registro=empleado)

        return Response(
            self.get_serializer(encomiendas, many=True).data,
            status=status.HTTP_201_CREATED
        )

    # — Bulk estado: cambiar estado a multiples encomiendas —————
    @extend_schema(
        summary='Cambiar estado a multiples encomiendas',
        description='Cambia el estado de varias encomiendas. Reporta cuales tuvieron errores.',
        tags=['Encomiendas'],
    )
    @action(detail=False, methods=['patch'], url_path='bulk_estado')
    def bulk_estado(self, request):
        """
        PATCH /api/v1/encomiendas/bulk_estado/
        Body: {"ids": [1, 2, 3], "estado": "TR", "observacion": "..."}
        Procesa cada encomienda y reporta cuales tuvieron errores.
        """
        ids = request.data.get('ids', [])
        nuevo_estado = request.data.get('estado')
        observacion = request.data.get('observacion', '')

        # Validar que llegaron los campos requeridos
        if not ids:
            return Response(
                {'error': 'El campo ids es requerido y no puede estar vacio.'},
                status=status.HTTP_400_BAD_REQUEST
            )
        if not nuevo_estado:
            return Response(
                {'error': 'El campo estado es requerido.'},

```

```

        status=status.HTTP_400_BAD_REQUEST
    )

    try:
        empleado = Empleado.objects.get(email=request.user.email)
    except Empleado.DoesNotExist:
        return Response(
            {'error': 'El usuario no tiene un empleado asociado.'},
            status=status.HTTP_403_FORBIDDEN
        )

    encomiendas = Encomienda.objects.filter(id__in=ids)
    actualizadas = []
    errores = []

    for enc in encomiendas:
        try:
            enc.cambiar_estado(nuevo_estado, empleado, observacion)
            actualizadas.append(enc.id)
        except ValueError as e:
            errores.append({'id': enc.id, 'error': str(e)})

    # Ids que no existen
    ids_procesados = list(encomiendas.values_list('id', flat=True))
    no_encontrados = [i for i in ids if i not in ids_procesados]

    return Response({
        'actualizadas': actualizadas,
        'errores': errores,
        'no_encontrados': no_encontrados,
        'total': len(actualizadas),
    })

```

## Paso 7 — Reiniciar y verificar los endpoints bulk

Python

```
# Reiniciar el contenedor
```

```
docker compose restart web
```

```
# Verificar que Django no reporta errores
```

```
docker compose exec web python manage.py check
```

```
# — Probar bulk_create —————
```

```
# Necesitas los IDs de clientes y rutas existentes en tu base de datos
```

```
# Primero obtenlos:
```

```
curl http://localhost:8000/api/v1/clientes/ \
  -H "Authorization: Bearer $TOKEN" | python3 -m json.tool | grep '"id"'
```

```
curl http://localhost:8000/api/v1/rutas/ \
  -H "Authorization: Bearer $TOKEN" | python3 -m json.tool | grep '"id"'
```

```
# Crear 3 encomiendas en una sola peticion:
```

```
curl -X POST http://localhost:8000/api/v1/encomiendas/bulk_create/ \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '[
    {
      "codigo": "ENC-2026-BULK-001",
      "descripcion": "Primer paquete del lote",
      "peso_kg": "2.5",
      "remitante_id": 1,
      "destinatario_id": 2,
      "ruta_id": 1,
      "costo_envio": "25.00"
    },
    {
      "codigo": "ENC-2026-BULK-002",
      "descripcion": "Segundo paquete del lote",
      "peso_kg": "1.0",
      "remitante_id": 1,
      "destinatario_id": 2,
      "ruta_id": 1,
      "costo_envio": "25.00"
    }
  ]' | python3 -m json.tool
```

```
# Respuesta esperada (201 Created):
```

```
# [
```

```
# {"id": 10, "codigo": "ENC-2026-BULK-001", "estado": "PE", ...},
# {"id": 11, "codigo": "ENC-2026-BULK-002", "estado": "PE", ...}

# ]
```

Python

```
# — Probar bulk_estado —————
# Cambiar el estado de las encomiendas 10 y 11 a En Transito:
curl -X PATCH http://localhost:8000/api/v1/encomiendas/bulk_estado/ \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "ids": [10, 11],
    "estado": "TR",
    "observacion": "Lote recogido en agencia Lima"
  }' | python3 -m json.tool

# Respuesta esperada (200 OK):
# {
#   "actualizadas": [10, 11],
#   "errores": [],
#   "no_encontrados": [],
#   "total": 2
# }

# Probar con un ID que ya esta en TR (debe reportar error):
curl -X PATCH http://localhost:8000/api/v1/encomiendas/bulk_estado/ \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"ids": [10, 11, 99999], "estado": "TR"}' | python3 -m json.tool

# Respuesta esperada:
# {
#   "actualizadas": [],
#   "errores": [
#     {"id": 10, "error": "La encomienda ya se encuentra en estado En transito"},
#     {"id": 11, "error": "La encomienda ya se encuentra en estado En transito"}
#   ]
# }
```

```
# ],
# "no_encontrados": [99999],
# "total": 0
# }
```

## Paso 8 — Verificar desde el shell de Django

Python

### Verificar desde el shell de Django

```
docker compose exec web python manage.py shell
```

```
from envios.serializers import EncomiendaSerializer
from clientes.models import Cliente
from rutas.models import Ruta
from envios.models import Empleado, Encomienda
```

```
c1 = Cliente.objects.first()
c2 = Cliente.objects.exclude(pk=c1.pk).first()
r1 = Ruta.objects.first()
emp = Empleado.objects.first()
```

# — Verificar to\_representation —————

```
from django.test import RequestFactory
from django.contrib.auth.models import User
```

```
user = User.objects.filter(is_staff=False).first()
factory = RequestFactory()
request = factory.get('/')
request.user = user
```

```
enc = Encomienda.objects.con_relaciones().first()
s = EncomiendaSerializer(enc, context={'request': request})
```

```
print('campos en la respuesta:', list(s.data.keys()))
print('tiene estado_color:', 'estado_color' in s.data) # True
print('tiene ruta_codigo:', 'ruta_codigo' in s.data) # True
```

```

print('tiene costo_display:', 'costo_display' in s.data) # True
print('observaciones ocultas:', 'observaciones' not in s.data) # True (no-staff)

# — Verificar bulk con many=True —————
from django.db import connection, reset_queries
from django.conf import settings
settings.DEBUG = True
reset_queries()

data = [
    {'codigo': 'ENC-BULK-SH-001', 'descripcion': 'Test 1',
     'peso_kg': '1.0', 'remitente_id': c1.pk, 'destinatario_id': c2.pk,
     'ruta_id': r1.pk, 'costo_envio': '25.00'},
    {'codigo': 'ENC-BULK-SH-002', 'descripcion': 'Test 2',
     'peso_kg': '2.0', 'remitente_id': c1.pk, 'destinatario_id': c2.pk,
     'ruta_id': r1.pk, 'costo_envio': '25.00'},
]

s = EncomiendaSerializer(data=data, many=True)
s.is_valid(raise_exception=True)
encomiendas = s.save(empleado_registro=emp)

print(f'Creadas: {len(encomiendas)}')
print(f'Queries SQL usadas: {len(connection.queries)}')

# Queries SQL usadas: 1 <- una sola INSERT para todas

```

## RESUMEN — ARCHIVOS MODIFICADOS EN ESTA SECCIÓN

envios/serializers.py <- se agregaron:

- EncomiendaBulkSerializer con create() y update()
- list\_serializer\_class = EncomiendaBulkSerializer en Meta
- to\_representation() en EncomiendaSerializer:
  - campos ruta\_codigo, ruta\_destino, ruta\_origen
  - campo costo\_display (S/ 25.00)
  - campo estado\_color (gray/blue/orange/green/red)
  - oculta observaciones y empleado\_registro para no-staff
- to\_internal\_value() en EncomiendaSerializer:
  - normaliza codigo a mayusculas
  - limpia espacios de descripcion
  - normaliza costo\_envio a 2 decimales

envios/viewsets.py <- se agregaron dos acciones al EncomiendaViewSet:

- bulk\_create: POST /api/v1/encomiendas/bulk\_create/
- bulk\_estado: PATCH /api/v1/encomiendas/bulk\_estado/

Nuevos endpoints disponibles:

POST /api/v1/encomiendas/bulk\_create/

Body: [{enc1}, {enc2}, ...]

Respuesta: [{...}, {...}] con status 201

PATCH /api/v1/encomiendas/bulk\_estado/

Body: {ids: [1,2,3], estado: 'TR', observacion: '...'}

Respuesta: {actualizadas: [...], errores: [...], no\_encontrados: [...]}

## 6.14. Optimización de Consultas — Problema N+1

El problema N+1 ocurre cuando se hacen N consultas adicionales para acceder a datos relacionados de N objetos. Con 100 encomiendas, acceder a `enc.remitente.nombre` dentro de un serializer genera 101 queries SQL en lugar de 2.

### Entender el problema con un ejemplo real

Primero veamos exactamente por qué ocurre el problema en nuestro proyecto. Cuando el serializer accede a campos de objetos relacionados, Django hace una consulta separada por cada objeto:

Python

#### El problema N+1 en SQL

```
# Lo que ocurre internamente cuando DRF serializa 50 encomiendas
# SIN select_related ni prefetch_related:
```

```
# Query 1: traer todas las encomiendas
```

```
SELECT * FROM encomiendas LIMIT 15;
```

```
# Query 2: traer el remitente de la encomienda 1
```

```
SELECT * FROM clientes WHERE id = 5;
```

```
# Query 3: traer el remitente de la encomienda 2
```

```
SELECT * FROM clientes WHERE id = 12;
```

```
# Query 4: traer el remitente de la encomienda 3
```

```
SELECT * FROM clientes WHERE id = 5;  # repetida!
```

```
# ... y así hasta 15 queries más para los destinatarios ...
```

```
# ... y 15 más para las rutas ...
```

```
# ... y 15 más para los empleados ...
```

```
# TOTAL: 1 + 15 + 15 + 15 + 15 = 61 queries para 15 encomiendas
# Con 100 encomiendas seria: 1 + 100 + 100 + 100 + 100 = 401 queries
```

El `EncomiendaSerializer` actual expone campos de objetos relacionados:

Python

```
# envios/serializers.py
# Estos campos SON la causa del N+1:

class EncomiendaSerializer(serializers.ModelSerializer):
    estado_display = serializers.SerializerMethodField()
    esta_entregada = serializers.ReadOnlyField() # accede al modelo
    tiene_retraso = serializers.ReadOnlyField() # accede al modelo
    descripcion_corta = serializers.ReadOnlyField() # accede al modelo

    # ESTOS son los que causan N+1 si no hay select_related:
    # Al serializar, DRF accede a: enc.remitente, enc.destinatario,
    # enc.ruta, enc.empleado_registro -> 4 queries extra por objeto
```

## Paso 1 — Instalar django-silk para medir las queries

Instalamos `django-silk` dentro del contenedor Docker para visualizar exactamente cuántas queries se ejecutan por request:

None

```
# Agregar al requirements.txt
django-silk==5.1.0
```

```
None
# Reconstruir la imagen
docker compose down
docker compose build
docker compose up -d
```

```
None
# Verificar que se instaló

docker compose exec web pip show django-silk
```

## Paso 2 — Configurar django-silk en settings.py

Abre `config/settings.py` y agrega silk. Solo para desarrollo, nunca en producción:

```
Python

# config/settings.py
# Agregar solo si DEBUG = True

if DEBUG:
    INSTALLED_APPS += ['silk']
    MIDDLEWARE     += ['silk.middleware.SilkyMiddleware']
    SILKY_PYTHON_PROFILER = True

    SILKY_META = True    # mostrar queries de silk en el panel
```

### 6.14.4 Paso 3 — Registrar las URLs de silk

Abre `config/urls.py` y agrega la URL del panel de silk:

Python

```
# config/urls.py
from django.conf import settings

urlpatterns = [
    ...
    # Panel de silk (solo en desarrollo)
]

if settings.DEBUG:
    from silk import urls as silk_urls
    urlpatterns += [
        path('silk/', include('silk.urls', namespace='silk')),
    ]
    urlpatterns += static(settings.STATIC_URL,
document_root=settings.STATIC_ROOT)

    urlpatterns += static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT)
```

## Paso 4 — Aplicar la migración de silk y reiniciar

None

```
# silk necesita sus propias tablas en la BD
docker compose exec web python manage.py migrate

# Verificar que no hay errores
docker compose exec web python manage.py check

# Reiniciar el contenedor web
docker compose restart web

# Abrir el panel de silk en el navegador:
# http://localhost:8000/silk/
```

```
# (requiere estar logueado como superusuario)
```

## Paso 5 — Demostrar el problema N+1

Primero vamos a **reproducir el problema** quitando el `con_relaciones()` del ViewSet temporalmente para ver el impacto en el número de queries:

```
Python
# envios/viewsets.py
# TEMPORAL para demostrar el problema - NO dejar así en el proyecto

class EncomiendaViewSet(viewsets.ModelViewSet):
    # Sin con_relaciones(): Django hace queries separadas por cada FK
    queryset = Encomienda.objects.all()    # <- SIN OPTIMIZAR
```

Con el ViewSet sin optimizar, haz una petición al listado y observa silk:

```
Python

Demostrar el problema con silk y curl
# Obtener el token JWT
curl -X POST http://localhost:8000/api/v1/auth/token/ \
  -H 'Content-Type: application/json' \
  -d '{"username": "admin", "password": "tu_password"}'

# Guardar el token
TOKEN="eyJhbGciOiJIUzI1NiIsInR5cCI6I..".

# Hacer la petición al listado
curl http://localhost:8000/api/v1/encomiendas/ \
  -H "Authorization: Bearer $TOKEN"

# Abrir el panel de silk:
# http://localhost:8000/silk/requests/
# Buscar el request GET /api/v1/encomiendas/
```

```
# Hacer clic en el numero de queries
# Veras algo como: 61 queries para 15 encomiendas

# También puedes verificar desde el shell de Django:
docker compose exec web python manage.py shell
```

None

```
# Dentro del shell: contar queries manualmente
from django.db import connection, reset_queries
from django.conf import settings
settings.DEBUG = True # necesario para registrar queries

from envios.models import Encomienda
from envios.serializers import EncomiendaSerializer

# — SIN OPTIMIZAR —————
reset_queries()
encomiendas = list(Encomienda.objects.all()[:15]) # sin con_relaciones()
for enc in encomiendas:
    # Simular lo que hace el serializer al acceder a FKs
    _ = enc.remitente.nombre_completo
    _ = enc.destinatario.nombre_completo
    _ = enc.ruta.destino
    _ = enc.empleado_registro.apellidos

print(f'Queries ejecutadas: {len(connection.queries)}')
# Resultado esperado: 61 queries (1 + 15*4)

# Ver las primeras 5 queries (todas iguales, solo cambia el id)
for q in connection.queries[:5]:
    print(q['sql'][:80])
# SELECT * FROM clientes WHERE id = 5
# SELECT * FROM clientes WHERE id = 12

# SELECT * FROM clientes WHERE id = 5 <- repetida!
```

## Paso 6 — Aplicar la solución con `con_relaciones()`

El manager `con_relaciones()` ya fue implementado en la Sesión 3 dentro de `envios/querysets.py`. Ahora lo conectamos correctamente al ViewSet:

`envios/querysets.py` — ya existe, solo revisar

Python

```
# envios/querysets.py
# Este archivo ya existe desde la Sesión 3 con este contenido:

class EncomiendaQuerySet(models.QuerySet):

    def con_relaciones(self):
        """
        select_related -> hace JOIN para ForeignKeys (1 sola query con JOIN)
        prefetch_related -> hace query separada para relaciones inversas

        Sin esto: 1 + N*4 queries (N = numero de encomiendas)
        Con esto: 2 queries siempre, sin importar cuantas encomiendas haya
        """
        return self.select_related(
            'remitente',          # JOIN con tabla clientes (remitente)
            'destinatario',       # JOIN con tabla clientes (destinatario)
            'ruta',               # JOIN con tabla rutas
            'empleado_registro',  # JOIN con tabla empleados
        ).prefetch_related(
            'historial',          # query separada: SELECT ... WHERE encomienda_id
IN (...)
            'historial__empleado', # prefetch anidado del empleado del historial
        )

    def pendientes(self): return self.filter(estado='PE')
    def en_transito(self): return self.filter(estado='TR')
    def activas(self): return self.filter(estado__in=['PE', 'TR', 'DE'])
    def con_retraso(self):
        from django.utils import timezone
        return self.activas().filter(fecha_entrega_est__lt=timezone.now().date())
```

Ahora actualiza `envios/viewsets.py` para que TODAS las acciones usen el queryset optimizado:

Python

```
# envios/viewsets.py
# RESTAURAR: usar con_relaciones() en lugar de .all()

class EncomiendaViewSet(viewsets.ModelViewSet):
    # El queryset base siempre usa con_relaciones()
    queryset = Encomienda.objects.con_relaciones()
    serializer_class = EncomiendaSerializer
    permission_classes = [IsAuthenticated]
    pagination_class = EncomiendaPagination

    def get_queryset(self):
        """
        Todas las acciones del ViewSet pasan por aqui.
        Siempre empezamos con con_relaciones() para evitar N+1.
        Los filtros adicionales se aplican encima.
        """
        qs = Encomienda.objects.con_relaciones()

        # Filtros opcionales (se agregan en tema 6.4)
        estado = self.request.query_params.get('estado')
        if estado:
            qs = qs.filter(estado=estado)

        q = self.request.query_params.get('search')
        if q:
            from django.db.models import Q
            qs = qs.filter(
                Q(codigo__icontains=q) |
                Q(remitente__apellidos__icontains=q) |
                Q(destinatario__apellidos__icontains=q)
            )
        return qs

    # Las acciones personalizadas tambien usan con_relaciones()
    @action(detail=False, methods=['get'])
    def con_retraso(self, request):
        qs = Encomienda.objects.con_retraso().con_relaciones()
        return Response(self.get_serializer(qs, many=True).data)
```

```

@action(detail=False, methods=['get'])
def pendientes(self, request):
    qs = Encomienda.objects.pendientes().con_relaciones()

    return Response(self.get_serializer(qs, many=True).data)

```

## Paso 7 — Verificar la mejora con el shell

Django Shell — verificar queries con optimización

Python

```

# Dentro del shell de Django:
docker compose exec web python manage.py shell

from django.db import connection, reset_queries
from django.conf import settings
settings.DEBUG = True

from envios.models import Encomienda

# — CON OPTIMIZACION —————
reset_queries()
encomiendas = list(Encomienda.objects.con_relaciones()[:15])
for enc in encomiendas:
    _ = enc.remitente.nombre_completo           # sin query extra
    _ = enc.destinatario.nombre_completo        # sin query extra
    _ = enc.ruta.destino                        # sin query extra
    _ = enc.empleado_registro.apellidos         # sin query extra
    _ = list(enc.historial.all())                # sin query extra

print(f'Queries ejecutadas: {len(connection.queries)}')
# Resultado esperado: 2 queries
# Query 1: SELECT encomiendas con JOINS (select_related)
# Query 2: SELECT historial WHERE encomienda_id IN (...) (prefetch_related)

# Ver exactamente las queries ejecutadas:

```

```

for i, q in enumerate(connection.queries, 1):
    print(f'--- Query {i} ---')
    print(q['sql'][:120])
    print(f'Tiempo: {q["time"]}s')

# Salida esperada:
# --- Query 1 ---
# SELECT encomiendas.*, clientes_r.*, clientes_d.*, rutas.*, empleados.*
# FROM encomiendas
# INNER JOIN clientes AS clientes_r ON (encomiendas.remitente_id = clientes_r.id)
# INNER JOIN clientes AS clientes_d ON (encomiendas.destinatario_id =
# clientes_d.id)
# INNER JOIN rutas ON (encomiendas.ruta_id = rutas.id)
# INNER JOIN empleados ON (encomiendas.empleado_registro_id = empleados.id)
# LIMIT 15
# Tiempo: 0.003s

# --- Query 2 ---
# SELECT historial_estados.* FROM historial_estados
# WHERE historial_estados.encomienda_id IN (1, 2, 3, ..., 15)
# Tiempo: 0.001s

```

## Paso 8 — Verificar con silk en el navegador

Python

```
# 1. Hacer la petición con el token
curl http://localhost:8000/api/v1/encomiendas/ \
  -H "Authorization: Bearer $TOKEN"

# 2. Abrir el panel de silk:
#     http://localhost:8000/silk/requests/

# 3. Hacer clic en el ultimo request GET /api/v1/encomiendas/

# 4. Observar la mejora:
#     Antes (sin optimizar): 61 queries / ~120ms
#     Despues (con_relaciones): 2 queries / ~8ms

# 5. Hacer clic en 'SQL' para ver las 2 queries con sus JOINS
```

## Paso 9 — Optimizar con only() para el listado

El listado de encomiendas muestra menos campos que el detalle. Con `only()` traemos solo los campos que el serializer del listado necesita, reduciendo la cantidad de datos transferidos desde la BD:

Python

```
# envios/serializers.py
# Agregar un serializer ligero para el listado

class EncomiendaListSerializer(serializers.ModelSerializer):
    """
    Serializer optimizado para el listado.
    Solo los campos necesarios para mostrar la tabla.
    """
```

```

    No incluye descripcion larga, observaciones ni historial.
    """
    remitente_nombre =
serializers.ReadOnlyField(source='remitente.nombre_completo')
    destinatario_nombre =
serializers.ReadOnlyField(source='destinatario.nombre_completo')
    ruta_destino = serializers.ReadOnlyField(source='ruta.destino')
    estado_display = serializers.SerializerMethodField()
    tiene_retraso = serializers.ReadOnlyField()

class Meta:
    model = Encomienda
    fields = [
        'id', 'codigo', 'estado', 'estado_display',
        'remitente_nombre', 'destinatario_nombre',
        'ruta_destino', 'peso_kg', 'costo_envio',
        'fecha_registro', 'fecha_entrega_est', 'tiene_retraso',
    ]

    def get_estado_display(self, obj):
    return obj.get_estado_display()

    read_only_fields = ['articulo_id']

```

Ahora actualiza el ViewSet para usar el serializer ligero en el listado y `only()` para traer solo los campos necesarios:

Python

```

# envios/viewsets.py
from .serializers import (
    EncomiendaSerializer,
    EncomiendaListSerializer,
    EncomiendaDetailSerializer,
    HistorialEstadoSerializer,
)

class EncomiendaViewSet(viewsets.ModelViewSet):

```

```

queryset          = Encomienda.objects.con_relaciones()
permission_classes = [IsAuthenticated]
pagination_class   = EncomiendaPagination

def get_serializer_class(self):
    """Serializer diferente segun la accion"""
    if self.action == 'list':
        return EncomiendaListSerializer    # ligero: solo campos del listado
    if self.action == 'retrieve':
        return EncomiendaDetailSerializer  # completo: con objetos anidados
    return EncomiendaSerializer            # estandar: para crear/editar

def get_queryset(self):
    qs = Encomienda.objects.con_relaciones()

    # Para el listado: traer solo los campos del EncomiendaListSerializer
    # Esto reduce el volumen de datos en la query SQL
    if self.action == 'list':
        qs = qs.only(
            # Campos propios de encomienda
            'id', 'codigo', 'estado',
            'peso_kg', 'costo_envio',
            'fecha_registro', 'fecha_entrega_est',
            # Campos del remitente (necesita el prefijo del related)
            'remitente__nombres', 'remitente__apellidos',
            # Campos del destinatario
            'destinatario__nombres', 'destinatario__apellidos',
            # Campos de la ruta
            'ruta__destino',
        )

    return qs

```

## Paso 10 — Comparativa final de rendimiento

Python

```
# Dentro del shell: comparativa completa
docker compose exec web python manage.py shell

from django.db import connection, reset_queries
from django.conf import settings
import time
settings.DEBUG = True

from envios.models import Encomienda

print('=' * 60)
print('CASO 1: Sin ninguna optimizacion')
print('=' * 60)
reset_queries()
t0 = time.time()
encomiendas = list(Encomienda.objects.all()[:15])
for enc in encomiendas:
    _ = enc.remitente.nombre_completo
    _ = enc.destinatario.nombre_completo
    _ = enc.ruta.destino
print(f'Queries: {len(connection.queries)}')
print(f'Tiempo: {(time.time()-t0)*1000:.1f}ms')

print()
print('=' * 60)
print('CASO 2: Con con_relaciones() (select_related + prefetch_related)')
print('=' * 60)
reset_queries()
t0 = time.time()
encomiendas = list(Encomienda.objects.con_relaciones()[:15])
for enc in encomiendas:
    _ = enc.remitente.nombre_completo
    _ = enc.destinatario.nombre_completo
    _ = enc.ruta.destino
print(f'Queries: {len(connection.queries)}')
```

```

print(f'Tiempo:  {(time.time()-t0)*1000:.1f}ms')

print()
print('=' * 60)
print('CASO 3: Con con_relaciones() + only() (campos limitados)')
print('=' * 60)
reset_queries()
t0 = time.time()
encomiendas = list(
    Encomienda.objects.con_relaciones().only(
        'id', 'codigo', 'estado', 'peso_kg', 'costo_envio',
        'fecha_registro', 'fecha_entrega_est',
        'remitante__nombres', 'remitante__apellidos',
        'destinatario__nombres', 'destinatario__apellidos',
        'ruta__destino',
    )[:15]
)
for enc in encomiendas:
    _ = enc.remitente.nombre_completo
    _ = enc.destinatario.nombre_completo
    _ = enc.ruta.destino
print(f'Queries:  {len(connection.queries)}')
print(f'Tiempo:  {(time.time()-t0)*1000:.1f}ms')

# Salida esperada:
# =====
# CASO 1: Sin ninguna optimizacion
# =====
# Queries: 61
# Tiempo:  180.3ms
#
# =====
# CASO 2: Con con_relaciones()
# =====
# Queries: 2
# Tiempo:  12.4ms
#
# =====
# CASO 3: Con con_relaciones() + only()
# =====
# Queries: 2
# Tiempo:   7.1ms

```

## Paso 11 — Verificar el endpoint final con curl y silk

Python

```
# Reiniciar el contenedor para aplicar todos los cambios
docker compose restart web

# Hacer la petición al listado optimizado
curl http://localhost:8000/api/v1/encomiendas/ \
  -H "Authorization: Bearer $TOKEN" | python3 -m json.tool | head -30

# Respuesta esperada (paginada, con nombres del serializer ligero):
# {
#   "count": 50,
#   "next": "http://localhost:8000/api/v1/encomiendas/?page=2",
#   "previous": null,
#   "results": [
#     {
#       "id": 1,
#       "codigo": "ENC-2026-001",
#       "estado": "TR",
#       "estado_display": "En transito",
#       "remitante_nombre": "Ramirez Torres, Carlos",
#       "destinatario_nombre": "Flores Diaz, Ana",
#       "ruta_destino": "Trujillo",
#       "tiene_retraso": false,
#       ....
#     }
#   ]
# }

# Revisar en silk: http://localhost:8000/silk/requests/
# El request GET /api/v1/encomiendas/ debe mostrar:
# - Num queries: 2
# - Tiempo total: < 15ms

# Probar el detalle (usa EncomiendaDetailSerializer con objetos anidados):
curl http://localhost:8000/api/v1/encomiendas/1/ \
  -H "Authorization: Bearer $TOKEN" | python3 -m json.tool

# Debe mostrar remitente como objeto completo, historial, etc.
```

## Verificación final con curl y silk

Python

```
# Reiniciar el contenedor para aplicar todos los cambios
docker compose restart web

# Hacer la petición al listado optimizado
curl http://localhost:8000/api/v1/encomiendas/ \
  -H "Authorization: Bearer $TOKEN" | python3 -m json.tool | head -30

# Respuesta esperada (paginada, con nombres del serializer ligero):
# {
#   "count": 50,
#   "next": "http://localhost:8000/api/v1/encomiendas/?page=2",
#   "previous": null,
#   "results": [
#     {
#       "id": 1,
#       "codigo": "ENC-2026-001",
#       "estado": "TR",
#       "estado_display": "En transito",
#       "remitante_nombre": "Ramirez Torres, Carlos",
#       "destinatario_nombre": "Flores Diaz, Ana",
#       "ruta_destino": "Trujillo",
#       "tiene_retraso": false,
#       ....
#     }
#   ]
# }

# Revisar en silk: http://localhost:8000/silk/requests/
# El request GET /api/v1/encomiendas/ debe mostrar:
# - Num queries: 2
# - Tiempo total: < 15ms

# Probar el detalle (usa EncomiendaDetailSerializer con objetos anidados):
curl http://localhost:8000/api/v1/encomiendas/1/ \
  -H "Authorization: Bearer $TOKEN" | python3 -m json.tool

# Debe mostrar remitente como objeto completo, historial, etc.
```

## RESUMEN — ARCHIVOS MODIFICADOS EN ESTA SECCIÓN

requirements.txt      <- se agrego: django-silk==5.1.0

config/settings.py      <- se agrego bloque 'if DEBUG: silk...'

config/urls.py      <- se agrego ruta del panel silk (solo en DEBUG)

envios/querysets.py      <- sin cambios (con\_relaciones()) ya estaba de Sesion 3)  
solo verificar que tiene select\_related y prefetch\_related

envios/serializers.py      <- se agrego EncomiendaListSerializer (campos ligeros)

envios/viewsets.py      <- se actualizo get\_serializer\_class() para usar  
EncomiendaListSerializer en list,  
EncomiendaDetailSerializer en retrieve;  
se actualizo get\_queryset() con only() para list

### Resultado final:

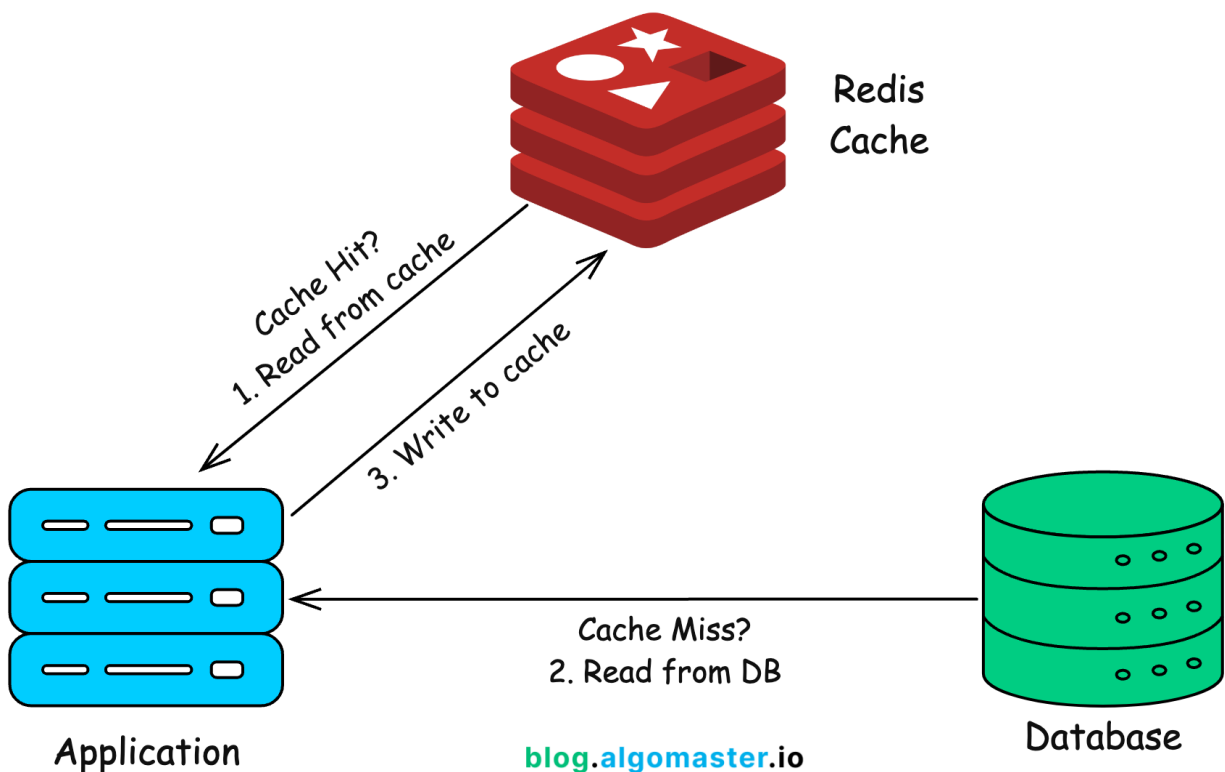
Listado: 2 queries en lugar de 61 (reduccion del 96%)

Detalle: 2 queries en lugar de 5

Tiempo respuesta: ~8ms en lugar de ~180ms

## 6.15. Caching con Redis

El caching almacena temporalmente los resultados de endpoints costosos para evitar consultas repetidas a la base de datos. Con Redis como backend, el caché es compartido entre todos los workers de la aplicación. Redis es un motor de almacenamiento en memoria, rápido y de código abierto, ideal para caché (almacenamiento en RAM) que mejora la velocidad de las aplicaciones y reduce la carga en bases de datos. Permite almacenar pares clave-valor, resultados de consultas, sesiones y otros datos de lectura frecuente, ofreciendo respuestas inferiores al milisegundo



## Configuración de Redis en Docker

Aquí vamos a necesitar confirmar el docker-compose.yml y el archivo [settings.py](#)

Python

```
# docker-compose.yml
services:
  web:
    ...
    depends_on:
      - db
      - redis

  db:
    image: postgres:15-alpine
    ...

  redis:
    image: redis:7-alpine
    ports:
      - '6379:6379'
    volumes:
      - redis_data:/data

volumes:
  postgres_data:

  redis_data:
```

Antes de modificar el archivo [settings.py](#) necesitamos instalar el paquete:

```
pip install django-redis
```

Python

```
# config/settings.py
CACHES = {
    'default': {
        'BACKEND': 'django_redis.cache.RedisCache',
        'LOCATION': 'redis://redis:6379/1',
        'OPTIONS': {
            'CLIENT_CLASS': 'django_redis.client.DefaultClient',
        }
    }
}

CACHE_TTL = 60 * 15 # 15 minutos por defecto
```

## Caching en ViewSets

Python

```
# envios/viewssets.py
from django.core.cache import cache
from django.utils.decorators import method_decorator
from django.views.decorators.cache import cache_page
from django.views.decorators.vary import vary_on_headers
from config.settings import CACHE_TTL

class RutaViewSet(viewsets.ReadOnlyModelViewSet):
    """Las rutas cambian poco - cachear el listado 15 minutos"""
    queryset = Ruta.objects.activas()
    serializer_class = RutaSerializer

    @method_decorator(cache_page(CACHE_TTL))
    @method_decorator(vary_on_headers('Authorization'))
    def list(self, request, *args, **kwargs):
        """Cache por usuario (vary_on_headers diferencia el token)"""
        return super().list(request, *args, **kwargs)
```

```

class EncomiendaViewSet(viewsets.ModelViewSet):

    # — Cache manual con invalidación —————
    @action(detail=False, methods=['get'], url_path='estadisticas')
    def estadisticas(self, request):
        """Estadísticas globales - se calculan cada 5 minutos"""
        cache_key = f'estadisticas_empleado_{request.user.id}'
        data = cache.get(cache_key)

        if data is None:
            # No está en caché: calcular y guardar
            data = {
                'activas': Encomienda.objects.activas().count(),
                'en_transito': Encomienda.objects.en_transito().count(),
                'con_retraso': Encomienda.objects.con_retraso().count(),
                'entregadas_mes': Encomienda.objects.filter(
                    estado='EN',
                    fecha_entrega_real__month=timezone.now().month
                ).count(),
            }
            cache.set(cache_key, data, CACHE_TTL) # guardar 15 min

        return Response(data)

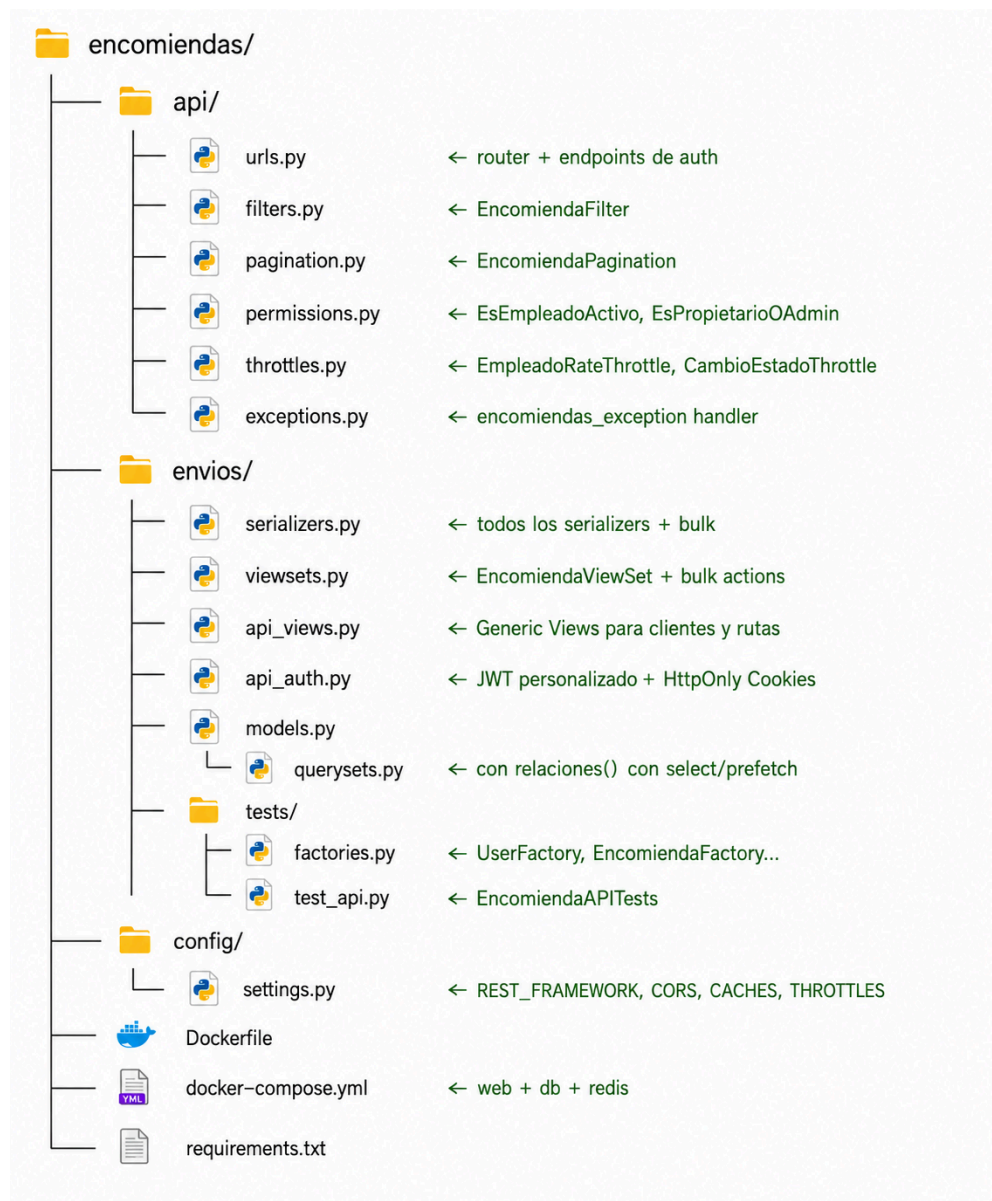
    def perform_update(self, serializer):
        """Invalidar caché cuando se actualiza una encomienda"""
        super().perform_update(serializer)
        # Borrar el caché de estadísticas de este empleado
        cache_key = f'estadisticas_empleado_{self.request.user.id}'
        cache.delete(cache_key)

    @action(detail=True, methods=['post'], url_path='cambiar_estado')
    def cambiar_estado(self, request, pk=None):
        enc = self.get_object()
        # ... lógica ...
        # Invalidar caché de estadísticas al cambiar un estado
        cache.delete_many([
            f'estadisticas_empleado_{request.user.id}',
            f'encomienda_detalle_{pk}',
        ])

        return Response(EncomiendaSerializer(enc).data)

```

# Estructura Final del Proyecto con DRF Completo



## ENTREGABLE FINAL — SESIONES 5 Y 6: DJANGO REST FRAMEWORK COMPLETO

Al finalizar debes poder demostrar:

### Sesión 5 — DRF #1: Fundamentos

1. Instalación y configuración de DRF en el proyecto de encomiendas.
2. EncomiendaSerializer y EncomiendaDetailSerializer con @property del modelo.
3. Endpoints FBV con @api\_view para listar y detalle de encomiendas.
4. Endpoints CBV con APIView con métodos get/post/patch/delete.
5. Mixins combinados para List+Create y Retrieve+Update+Destroy.
6. Generic Views: ListCreateAPIView y RetrieveUpdateDestroyAPIView.

### Sesión 6 — DRF #2: Avanzado

7. EncomiendaViewSet con ModelViewSet y @action para cambiar\_estado.
8. Acciones de lista: con\_retraso, pendientes, estadísticas.
9. Router registrado y URLs generadas automáticamente.
10. EncomiendaDetailSerializer con objetos anidados y historial.
11. Paginación: PageNumberPagination con page\_size=15.
12. Filtros: estado, búsqueda de texto, ordenamiento y con\_retraso.
13. JWT: obtención, uso en cabecera, renovación y payload personalizado.
14. Permisos: EsEmpleadoActivo y EsPropietarioOAdmin funcionando.
15. Validaciones: validate\_peso\_kg, validate\_codigo y validate() cruzado.
16. Swagger en <http://localhost:8000/api/docs/> con @extend\_schema.
17. Versionado: /api/v1/ y /api/v2/ con serializer distinto.

### Temas adicionales

18. Tests: al menos 6 tests que cubren list, create, error 400 y cambiar\_estado.
19. Throttling: EmpleadoRateThrottle en el ViewSet y LoginRateThrottle en auth.
20. Exception handler devuelve el mismo formato JSON en todos los errores.
21. CORS configurado con CORS\_ALLOWED\_ORIGINS.
22. to\_representation() personaliza la salida ocultando campos según el usuario.
23. Bulk: POST /api/v1/encomiendas/bulk\_create/ y PATCH /bulk\_estado/ funcionan.
24. select\_related y prefetch\_related en con\_relaciones(): máximo 3 queries por lista.
25. Redis en docker-compose.yml: endpoint /estadisticas/ cacheado 15 minutos.

Repositorio en GitHub con todos los archivos, tests pasando y README actualizado.

—